

Bahria University Islamabad

Department of Computer Science
Faculty of Computing and Information Technology

SkillLink

Freelance Marketplace System

A Complete Database Management System Project

Complete System Report

Covering Database Design, SQL Implementation,
Backend API, Frontend Interface, and Full Documentation

Course Title: Database Management System
Course Code: CSC 220
Instructor: Ms. Mehwish Pervaiz
Submission Date: April 30, 2026
Academic Year: 2025 – 2026

Group Members

Name	Enrollment No.
Zairish Fatima	01-135241-052
Maryam Qadir	01-135241-028
Saim Zafar	01-135241-045

Abstract

SkillLink is a full-stack freelance marketplace system built as part of the Database Management Systems course (CSC 220) at Bahria University Islamabad. It models the complete lifecycle of freelance work, starting from user registration and role-based specialization, then moving through project posting, competitive bidding, contract creation, milestone-based payment processing, and finally mutual review submission. The system runs on a three-layer architecture: an Oracle XE 21c relational database, a Node.js and Express backend API, and a React.js frontend interface.

At the database level, the system has 15 fully normalized tables. It uses key ER concepts including disjoint-total specialization where the Users entity splits into Client and Freelancer subtypes, weak entities for Bids and Reviews, and many-to-many relationships resolved through junction tables such as User_Skills and Project_Category. All tables satisfy the Third Normal Form (3NF), removing repeating groups, partial dependencies, and transitive dependencies. Referential integrity is enforced throughout using primary keys, foreign keys, unique constraints, and check constraints written directly in Oracle SQL.

The backend exposes 35 API endpoints spread across ten route files, each handling a specific group of Oracle tables. Three critical multi-table transactions protect data consistency: user registration, bid acceptance, and payment recording. Role-based access control works at three levels, keeping the capabilities of clients, freelancers, and administrators separate. The frontend gives a clean interface for all three roles, with an admin panel providing read-only access to all 15 tables.

This report covers every layer of the system in full detail. It includes the ER design, the relational data model, the complete SQL implementation with all DDL and DML scripts, the SQL queries showing joins and aggregation, the backend API structure, the frontend interface with screenshots, and a full API reference.

Keywords: Relational Database, Oracle XE, ER Diagram, Normalization, SQL Joins, Specialization, Weak Entities, REST API, Node.js, React.js, Role-Based Access Control

Contents

Abstract	1
List of Tables	10
1 Introduction	11
1.1 Project Overview	11
1.2 Problem Statement	12
1.2.1 Problems with Unstructured Systems	12
1.2.2 How SkillLink Solves These Problems	12
1.3 Objectives	13
1.4 Technology Stack	13
1.5 Scope of the Project	13
1.5.1 What the System Does	14
1.5.2 What the System Does Not Do	14
1.6 Report Structure	15
2 System Architecture	16
2.1 Overview	16
2.2 Database Layer	17
2.3 Backend Layer	17
2.4 Frontend Layer	18
2.5 Data Flow	18
2.6 Role-Based Access Control	19
3 ER Diagram and Conceptual Design	20
3.1 Overview	20
3.2 ER Concepts Applied	20
3.2.1 Specialization	20
3.2.2 Weak Entities	21
3.2.3 Many-to-Many Relationships	22

3.3	All Entities and Their Attributes	22
3.4	All Relationships and Cardinalities	24
3.5	ER Diagram	25
4	Relational Data Model	27
4.1	Overview	27
4.2	Conversion Rules Applied	27
4.3	Entity Classification	28
4.4	All 15 Tables	28
4.4.1	users	28
4.4.2	client_profile	29
4.4.3	freelancer_profile	29
4.4.4	skills	29
4.4.5	user_skills	29
4.4.6	categories	29
4.4.7	projects	30
4.4.8	project_category	30
4.4.9	project_files	30
4.4.10	bids	30
4.4.11	contracts	30
4.4.12	payments	31
4.4.13	reviews	31
4.4.14	disputes	31
4.4.15	notifications	31
4.5	Full RDM Schema	31
5	Normalization	37
5.1	Overview	37
5.2	First Normal Form (1NF)	37
5.2.1	Rule	37
5.2.2	Violation Example	38
5.2.3	How SkillLink Satisfies 1NF	38
5.3	Second Normal Form (2NF)	38
5.3.1	Rule	39
5.3.2	Violation Example	39
5.3.3	How SkillLink Satisfies 2NF	39
5.4	Third Normal Form (3NF)	40
5.4.1	Rule	40

5.4.2	Violation Example	40
5.4.3	How SkillLink Satisfies 3NF	40
6	Physical Implementation	42
6.1	Overview	42
6.2	Table Creation Order	42
6.3	Sequences	42
6.4	CREATE TABLE Statements	43
6.4.1	users	43
6.4.2	client_profile	44
6.4.3	freelancer_profile	44
6.4.4	skills	44
6.4.5	user_skills	45
6.4.6	categories	45
6.4.7	projects	45
6.4.8	project_category	45
6.4.9	project_files	46
6.4.10	bids	46
6.4.11	contracts	46
6.4.12	payments	47
6.4.13	reviews	47
6.4.14	disputes	47
6.4.15	notifications	48
6.5	Cleanup Script	48
7	Data Insertion	50
7.1	Overview	50
7.2	Insertion Order	50
7.3	Sample Data	50
7.3.1	users	50
7.3.2	client_profile	51
7.3.3	freelancer_profile	52
7.3.4	skills	52
7.3.5	user_skills	53
7.3.6	categories	53
7.3.7	projects	53
7.3.8	project_category	54
7.3.9	project_files	54

7.3.10	bids	54
7.3.11	contracts	55
7.3.12	payments	55
7.3.13	reviews	55
7.3.14	disputes	56
7.3.15	notifications	56
8	SQL Queries	57
8.1	Overview	57
8.2	Query 1 — All Projects with Client Name	57
8.3	Query 2 — All Bids with Freelancer Name and Project Title	58
8.4	Query 3 — Active Contracts with Client and Freelancer Names	58
8.5	Query 4 — Total Payments Received per Freelancer	58
8.6	Query 5 — Number of Bids per Project	59
8.7	Query 6 — Top Rated Freelancers	59
8.8	Query 7 — Projects with No Bids (LEFT JOIN)	60
8.9	Query 8 — Average Bid Amount per Project	60
8.10	Query 9 — All Skills of Each Freelancer	60
8.11	Query 10 — Clients Who Have Spent More Than 20000	61
8.12	Query 11 — All Reviews with Reviewer and Reviewee Names	61
8.13	Query 12 — Freelancers Who Have Never Won a Contract	62
8.14	Query 13 — Payment Status Summary	62
8.15	Query 14 — Open Disputes with Contract and User Details	62
8.16	Query 15 — Full Project Summary	63
9	Backend Architecture	64
9.1	Overview	64
9.2	Server Setup	64
9.3	Database Connection Helper	65
9.4	Authentication Middleware	66
9.5	Route File: auth.js	66
9.6	Route File: projects.js	68
9.7	Route File: bids.js	71
9.8	Route File: contracts.js	74
9.9	Route File: payments.js	76
9.10	Route File: reviews.js	78
9.11	Route File: disputes.js	80
9.12	Route File: notifications.js	81

9.13 Route File: admin.js	82
9.14 API Endpoints Summary	85
10 Frontend Architecture	87
10.1 Overview	87
10.2 Project Structure	87
10.3 Axios Configuration	88
10.4 Authentication Context	89
10.5 Protected Route Component	89
10.6 App.jsx — Route Definitions	90
10.7 Login Page	93
10.8 Client Pages	94
10.8.1 PostProject.jsx	94
10.8.2 ViewBids.jsx	95
10.8.3 MakePayment.jsx	95
10.9 Freelancer Pages	96
10.9.1 BrowseProjects.jsx	96
10.9.2 SubmitBid.jsx	97
10.9.3 Earnings.jsx	98
10.10 Admin Pages	99
10.10.1 SystemSummary.jsx	99
10.10.2 AllUsers.jsx	100
10.11 Role-Based Access Summary	101
11 User Interface	102
11.1 Overview	102
11.2 Authentication Pages	102
11.2.1 Login Page	102
11.2.2 Register Page	103
11.3 Client Pages	103
11.3.1 Client Dashboard	103
11.3.2 Post Project	104
11.3.3 View Bids	104
11.3.4 My Contracts (Client)	105
11.3.5 Make Payment	105
11.4 Freelancer Pages	106
11.4.1 Freelancer Dashboard	106
11.4.2 Browse Projects	106

11.4.3	Submit Bid	107
11.4.4	My Bids	107
11.4.5	Earnings	108
11.5	Admin Pages	108
11.5.1	Admin Dashboard	108
11.5.2	All Users	109
11.5.3	All Projects	109
11.5.4	System Summary	110
12	Conclusion and Future Work	111
12.1	Conclusion	111
12.2	Challenges Faced	112
12.2.1	Oracle Sequence Management	112
12.2.2	Foreign Key Insertion Order	112
12.2.3	Multi-Table Transactions in Node.js	112
12.2.4	JWT Role Detection at Login	112
12.2.5	Long SQL Lines in LaTeX	113
12.3	Future Enhancements	113
12.4	Learning Outcomes	114

List of Tables

1.1	SkillLink Technology Stack	14
2.1	Three-Layer Architecture Summary	16
2.2	Role Permissions Summary	19
3.1	Many-to-Many Relationships and Junction Tables	22
3.2	All Entities and Attributes	22
3.3	All Relationships and Cardinalities	24
4.1	Entity Classification	28
4.2	users table	29
4.3	client_profile table	29
4.4	freelancer_profile table	30
4.5	skills table	30
4.6	user_skills table	31
4.7	categories table	31
4.8	projects table	32
4.9	project_category table	32
4.10	project_files table	33
4.11	bids table	33
4.12	contracts table	34
4.13	payments table	34
4.14	reviews table	34
4.15	disputes table	35
4.16	notifications table	35
5.1	Before 1NF — skills stored as a list inside one column	38
5.2	After 1NF — user_skills junction table	38
5.3	Before 2NF — partial dependency on project_id only	39
5.4	After 2NF — bids table contains only bid-specific data	39
5.5	Before 3NF — transitive dependency between city and country	40

6.1 Table Creation Order 43

7.1 Data Insertion Order 51

9.1 All 35 API Endpoints 86

10.1 Frontend Pages by Role 101

12.1 Planned Future Enhancements 113

Chapter 1

Introduction

1.1 Project Overview

SkillLink is a relational database project that models how a freelancing platform works at the data level. The name comes from the core idea of the system: linking people who have skills with people who need those skills. In practice, a client who needs a logo designed posts a project, graphic designers submit bids with their proposed price and delivery time, the client selects one, a contract is created, payment is made, and both sides leave a review.

This is exactly how real platforms like Upwork and Fiverr operate. SkillLink models that entire workflow as a database system. There is no simplified or artificial scenario here. Every table exists because something real needs to be stored, and every relationship exists because two things in the real world are actually connected.

The project was chosen because it naturally fits almost every concept covered in a Semester 5 DBMS course into one coherent system. Specialization, weak entities, many-to-many relationships, normalization to 3NF, complex joins, aggregate queries, and role-based data access are all present and meaningful in this design. Beyond the database itself, the project was extended into a complete three-layer system with a Node.js backend API and a React.js frontend interface, fulfilling the bonus interface requirement of the assignment.

Quick Summary: SkillLink is a freelance marketplace with 15 Oracle SQL tables, a Node.js REST API with 35 endpoints, and a React.js frontend supporting three user roles: Client, Freelancer, and Admin.

1.2 Problem Statement

Before structured database systems, freelancing interactions were typically handled through email threads, spreadsheets, or messaging apps. These approaches have several problems that SkillLink is designed to solve.

1.2.1 Problems with Unstructured Systems

Data loss and duplication. Without a central database, records of projects, bids, contracts, and payments are scattered across files and inboxes. The same information gets copied into multiple places and versions become inconsistent. When a file is lost or a conversation becomes too long, information disappears permanently.

No audit trail. Manual systems have no way to track the status of a bid, the progress of a contract, or the history of payments in a reliable way. If a dispute arises between a client and a freelancer, there is no trustworthy record of what was agreed, what was paid, and when.

Inefficient reporting. Finding which freelancers worked on which projects, calculating total earnings, or identifying the most active clients requires manually scanning records. There is no way to generate a meaningful report without compiling data by hand from multiple sources.

No integrity enforcement. Nothing stops a freelancer from bidding twice on the same project, a payment from being recorded against the wrong contract, or a review from being submitted before a contract exists. Rules that should be automatic have to be verified manually.

1.2.2 How SkillLink Solves These Problems

SkillLink replaces all of this with a fully relational database where:

- All records are stored in clearly defined tables with enforced constraints, eliminating data loss and duplication
- Every record is timestamped and linked through foreign keys, creating a complete and verifiable audit trail for every interaction in the system
- SQL queries replace manual searching. Retrieving top-rated freelancers, total earnings, or most active clients takes a single statement
- Business rules are enforced at the database level: a freelancer cannot bid twice on

the same project (UNIQUE constraint on `project_id`, `freelancer_id`), a payment cannot reference a non-existent contract (foreign key), and a rating must be between 1 and 5 (CHECK constraint)

1.3 Objectives

The goals of this project are:

1. Design a complete relational database for a freelance marketplace with 15 tables covering all entities in the workflow
2. Apply ER concepts correctly: specialization (disjoint, total), weak entities, and M:N relationships resolved through junction tables
3. Convert the ER diagram into a full Relational Data Model (RDM) with all tables, columns, and constraints defined
4. Implement all 15 tables in Oracle XE using DBeaver in the correct dependency order
5. Insert realistic sample data into all tables with at least 10 records per core table
6. Write SQL queries demonstrating INNER JOIN, LEFT JOIN, aggregate functions (SUM, COUNT, AVG, MAX), and role-based conditional data retrieval
7. Normalize the database to Third Normal Form (3NF) and demonstrate it with examples from the actual tables
8. Develop a full-stack interface as a bonus: a Node.js backend API and a React.js frontend supporting Client, Freelancer, and Admin roles

1.4 Technology Stack

The complete technology stack used across all layers of the system is shown in Table 1.1.

1.5 Scope of the Project

Table 1.1: SkillLink Technology Stack

Layer	Technology	Purpose
Database	Oracle XE 21c	Primary relational database engine
DB Client	DBeaver Community	SQL editor and database browser
Query Language	Oracle SQL	DDL, DML, and data retrieval
Backend	Node.js + Express	REST API server and business logic
DB Driver	oracledb (npm)	Oracle connection pool and query execution
Authentication	JWT	Stateless token-based authentication
Frontend	React.js	Component-based user interface
HTTP Client	Axios	Frontend-to-backend HTTP requests
Routing	React Router DOM	Client-side navigation
Environment	dotenv	Secure environment variable management

1.5.1 What the System Does

- Stores all data related to users, projects, bids, contracts, payments, reviews, skills, categories, files, disputes, and notifications in 15 Oracle tables
- Enforces data integrity through primary keys, foreign keys, UNIQUE constraints, CHECK constraints, and NOT NULL requirements
- Supports complex data retrieval through SQL queries using joins, aggregation, filtering, and grouping
- Provides role-based access control separating the capabilities of clients, freelancers, and administrators
- Implements three atomic multi-table transactions to protect data consistency during critical operations
- Delivers a complete web interface for all three user roles through the React frontend

1.5.2 What the System Does Not Do

- It does not include a real-time payment gateway. Payments are recorded as data entries only

- It does not store actual files. The `project_files` table stores metadata like file name and upload date only
- It does not handle real-time notifications. The `notifications` table stores notification records for retrieval on demand
- It does not implement database triggers or stored procedures in this version. These are listed as future enhancements

1.6 Report Structure

This report is organized into four parts:

- **Part I – Database Design (Chapters 1–8):** Introduction, system architecture, ER diagram, relational data model, normalization, physical implementation, data insertion, and SQL queries
- **Part II – Backend Architecture (Chapters 9–14):** Node.js server setup, database connection, authentication, route structure, transactions, and admin queries
- **Part III – Frontend Architecture (Chapters 15–20):** React.js structure, authentication flow, role-based access, and screenshots of the interface
- **Part IV – Appendices:** Complete SQL scripts, ERD, RDM, and API reference

Chapter 2

System Architecture

2.1 Overview

SkillLink is built on a three-layer architecture. Each layer has a single responsibility and communicates with the layer directly next to it. The database layer stores and manages all data. The backend layer handles business logic and exposes data through an API. The frontend layer presents that data to the user and sends actions back to the backend.

This separation keeps the system clean and maintainable. A change in the frontend does not affect the database. A change in the database schema only requires updating the backend queries, not the user interface. In simple terms, each layer does one job and does it well.

Table 2.1: Three-Layer Architecture Summary

Layer	Technology	Responsibility
Database	Oracle XE 21c via DBEaver	Stores all data, enforces constraints and referential integrity
Backend	Node.js + Express	Processes requests, runs SQL queries, handles authentication and transactions
Frontend	React.js	Displays data to users, collects input, enforces role-based access in the UI

2.2 Database Layer

The database layer is the foundation of the entire system. It is built on Oracle XE 21c and managed through DBeaver Community Edition. All 15 tables live here along with their sequences, constraints, and relationships.

The database layer is responsible for:

- Storing every record in the system: users, projects, bids, contracts, payments, reviews, skills, categories, files, disputes, and notifications
- Enforcing referential integrity through foreign keys so that no orphan records can exist
- Enforcing business rules through constraints such as UNIQUE on bid entries and CHECK on status fields
- Generating unique IDs automatically through Oracle sequences
- Ensuring data consistency through atomic commits

No application logic lives at this layer. The database does not decide who can see what or what action should happen next. It simply stores data correctly and rejects data that violates its rules.

2.3 Backend Layer

The backend is a Node.js server using the Express framework. It sits between the database and the frontend and is the only layer that directly communicates with Oracle XE. The frontend never touches the database directly.

The backend is responsible for:

- Receiving HTTP requests from the frontend
- Verifying the identity of the user through JWT token validation
- Checking whether the user has permission to perform the requested action based on their role
- Running the appropriate Oracle SQL query using the oracledb connection pool
- Returning the result to the frontend as a JSON response

The backend has 10 route files, each handling a specific group of tables. Three of these routes involve multi-table transactions where multiple SQL statements must all succeed or all fail together. These transactions protect data consistency during user registration, bid acceptance, and payment recording.

2.4 Frontend Layer

The frontend is a React.js single-page application. It communicates with the backend exclusively through HTTP requests using Axios. The user never interacts with SQL or the database directly.

The frontend is responsible for:

- Presenting the correct interface based on the logged-in user's role (Client, Freelancer, or Admin)
- Sending user actions such as posting a project, submitting a bid, or recording a payment to the appropriate backend endpoint
- Storing the JWT token in application state after login and attaching it to every subsequent request
- Protecting routes so that a freelancer cannot access a client-only page and an unauthenticated user cannot access any protected page

2.5 Data Flow

The flow of data through the system follows the same path for every action a user takes. Here is how a typical request works, using the example of a freelancer submitting a bid:

1. The freelancer fills in the bid amount and delivery time on the frontend and clicks Submit
2. React sends a POST request to the backend endpoint `/api/bids` with the bid data and the JWT token in the request header
3. The backend middleware extracts the token, verifies it, and confirms the user is a freelancer

4. The backend runs an INSERT statement into the `bids` table in Oracle XE
5. Oracle XE checks the foreign keys (project must exist, freelancer must exist) and the UNIQUE constraint (this freelancer has not already bid on this project)
6. If all checks pass, the row is inserted and committed. If any check fails, Oracle rejects the insert and the backend returns an error
7. The backend sends a JSON response back to React indicating success or failure
8. React updates the interface to show the submitted bid or display the error message

Key Point: The frontend never writes to the database directly. The backend never skips constraint checking. All data passes through every layer in sequence, and each layer enforces its own rules independently.

2.6 Role-Based Access Control

The system supports three user roles. Each role has a different set of permissions enforced at both the backend and frontend levels.

Table 2.2: Role Permissions Summary

Role	Permissions
Client	Post projects, view bids on their projects, accept a bid, view their contracts, release payments, leave reviews, raise disputes
Freelancer	Browse open projects, submit bids, view their contracts, view payments received, leave reviews, view their notifications
Admin	Read-only access to all 15 tables. Can view all users, projects, bids, contracts, payments, reviews, disputes, and notifications across the entire system

At the backend level, each endpoint checks the role stored in the JWT token before executing any query. At the frontend level, React Router protects routes so that navigating to a restricted URL redirects the user back to their own dashboard.

Chapter 3

ER Diagram and Conceptual Design

3.1 Overview

The conceptual design is the first step in building any relational database. Before writing a single line of SQL, the system needs to be understood at a high level: what things exist in the domain, what information needs to be stored about each thing, and how different things relate to each other. This is captured in an Entity-Relationship (ER) diagram.

For SkillLink, the domain is a freelance marketplace. The real-world things that exist in this domain are users, projects, bids, contracts, payments, reviews, skills, categories, files, disputes, and notifications. Each of these becomes an entity in the ER diagram, and the connections between them become relationships.

3.2 ER Concepts Applied

The SkillLink ER diagram applies three key concepts from the extended ER model that go beyond basic entity-relationship modeling.

3.2.1 Specialization

Specialization is the process of dividing a general entity into more specific subtypes based on distinguishing characteristics. In SkillLink, the **USER** entity is the supertype. Every person using the system is a user, but not every user has the same role or the same attributes.

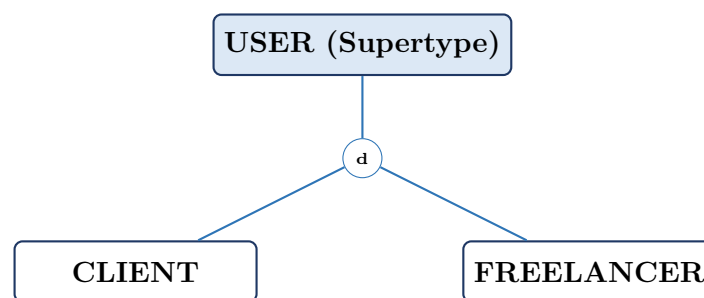
Users are divided into two subtypes:

- **CLIENT** — a user who posts projects and hires freelancers. Has additional attributes: `company_name` and `location`
- **FREELANCER** — a user who bids on projects and does the work. Has additional attributes: `bio`, `hourly_rate`, `rating`, `experience_level`, and `availability`

This specialization is both **disjoint** and **total**:

- **Disjoint** means a user can be either a client or a freelancer, but not both at the same time
- **Total** means every user must be one of the two subtypes. A user who is neither a client nor a freelancer cannot exist in the system

In the relational model, this is implemented using a shared primary key. The `client_id` in `client_profile` and the `freelancer_id` in `freelancer_profile` both reference `user_id` in the `users` table. This means a subtype record cannot exist without a corresponding user record.



3.2.2 Weak Entities

A weak entity is one that cannot be uniquely identified by its own attributes alone. It depends on another entity, called the identifying entity, for its existence and identification.

SkillLink has two weak entities:

BIDS is a weak entity because a bid has no meaning without a project and a freelancer. If the project is deleted, the bids on it have no reason to exist. The bid is identified by the combination of `project_id` and `freelancer_id` together, making this the partial key. In the relational model, a surrogate `bid_id` is used as the primary key for practical purposes, but the `UNIQUE` constraint on (`project_id`, `freelancer_id`) enforces the identifying relationship, ensuring one freelancer cannot bid twice on the same project.

REVIEWS is a weak entity because a review cannot exist without a contract. A review is always about a specific contract interaction between a specific client and

freelancer. If the contract is removed, the review has no context. The `contract_id` foreign key in reviews enforces this dependency.

3.2.3 Many-to-Many Relationships

A many-to-many (M:N) relationship exists when one record in a table can relate to many records in another table, and vice versa. Pure M:N relationships cannot be stored directly in a relational database and must be resolved using a junction table.

SkillLink has three M:N relationships:

Table 3.1: Many-to-Many Relationships and Junction Tables

Entity A	Entity B	Junction Table	Meaning
USER	SKILL	user_skills	One user can have many skills and one skill can belong to many users
PROJECT	CATEGORY	project_category	One project can belong to many categories and one category can have many projects
FREELANCER	PROJECT	bids	One freelancer can bid on many projects and one project can receive bids from many freelancers

3.3 All Entities and Their Attributes

The following table lists every entity in the SkillLink ER diagram along with its type and attributes.

Table 3.2: All Entities and Attributes

Entity	Type	Attributes
USER	Supertype	user_id (PK), name, email (unique), password, phone, created_at

Entity	Type	Attributes
CLIENT	Subtype	client_id (PK, FK), company_name, location
FREELANCER	Subtype	freelancer_id (PK, FK), bio, hourly_rate, rating, experience_level, availability
PROJECTS	Strong	project_id (PK), client_id (FK), title, description, budget, deadline, status
BIDS	Weak	bid_id (PK), project_id (FK), freelancer_id (FK), amount, delivery_time, status, submitted_at
CONTRACTS	Strong	contract_id (PK), bid_id (FK, unique), amount, start_date, end_date, status
PAYMENTS	Strong	payment_id (PK), contract_id (FK), amount, pay_date, method, status
REVIEWS	Weak	review_id (PK), contract_id (FK), reviewer_id (FK), reviewee_id (FK), rating, comment, created_at
SKILLS	Strong	skill_id (PK), name (unique), category
CATEGORIES	Strong	category_id (PK), name (unique), description
USER_SKILLS	Associative	user_id (FK), skill_id (FK)
PROJECT_CATEGORIES	Associative	project_id (FK), category_id (FK)
PROJECT_FILES	Strong	file_id (PK), project_id (FK), file_name, file_type, upload_date
DISPUTES	Strong	dispute_id (PK), contract_id (FK), raised_by (FK), reason, status, created_at
NOTIFICATIONS	Strong	notification_id (PK), user_id (FK), message, date_sent, is_read, type

3.4 All Relationships and Cardinalities

The following table lists every relationship between tables, including cardinality and whether participation is mandatory or optional on each side.

Table 3.3: All Relationships and Cardinalities

From	To	Type	Participation
users	client_profile	1:1	Optional on users, mandatory on client_profile
users	freelancer_profile	1:1	Optional on users, mandatory on freelancer_profile
users	user_skills	1:M	Optional on users, mandatory on user_skills
users	notifications	1:M	Optional on users, mandatory on notifications
users	reviews (reviewer)	1:M	Optional on users, mandatory on reviews
users	reviews (reviewee)	1:M	Optional on users, mandatory on reviews
users	disputes	1:M	Optional on users, mandatory on disputes
skills	user_skills	1:M	Optional on skills, mandatory on user_skills
client_profile	projects	1:M	Optional on client, mandatory on projects
projects	project_category	1:M	Optional on projects, mandatory on project_category
projects	project_files	1:M	Optional on projects, mandatory on project_files
projects	bids	1:M	Optional on projects, mandatory on bids
categories	project_category	1:M	Optional on categories, mandatory on project_category
freelancer_profile	bids	1:M	Optional on freelancer, mandatory on bids

From	To	Type	Participation
bids	contracts	1:1	Optional on bids, mandatory on contracts
contracts	payments	1:M	Optional on contracts, mandatory on payments
contracts	reviews	1:M	Optional on contracts, mandatory on reviews
contracts	disputes	1:1	Optional on contracts, mandatory on disputes

3.5 ER Diagram

The full ER diagram for SkillLink is shown below. It was drawn using draw.io and follows Chen notation with double rectangles for weak entities, diamond shapes for relationships, and the disjoint-total specialization symbol between USER and its subtypes.

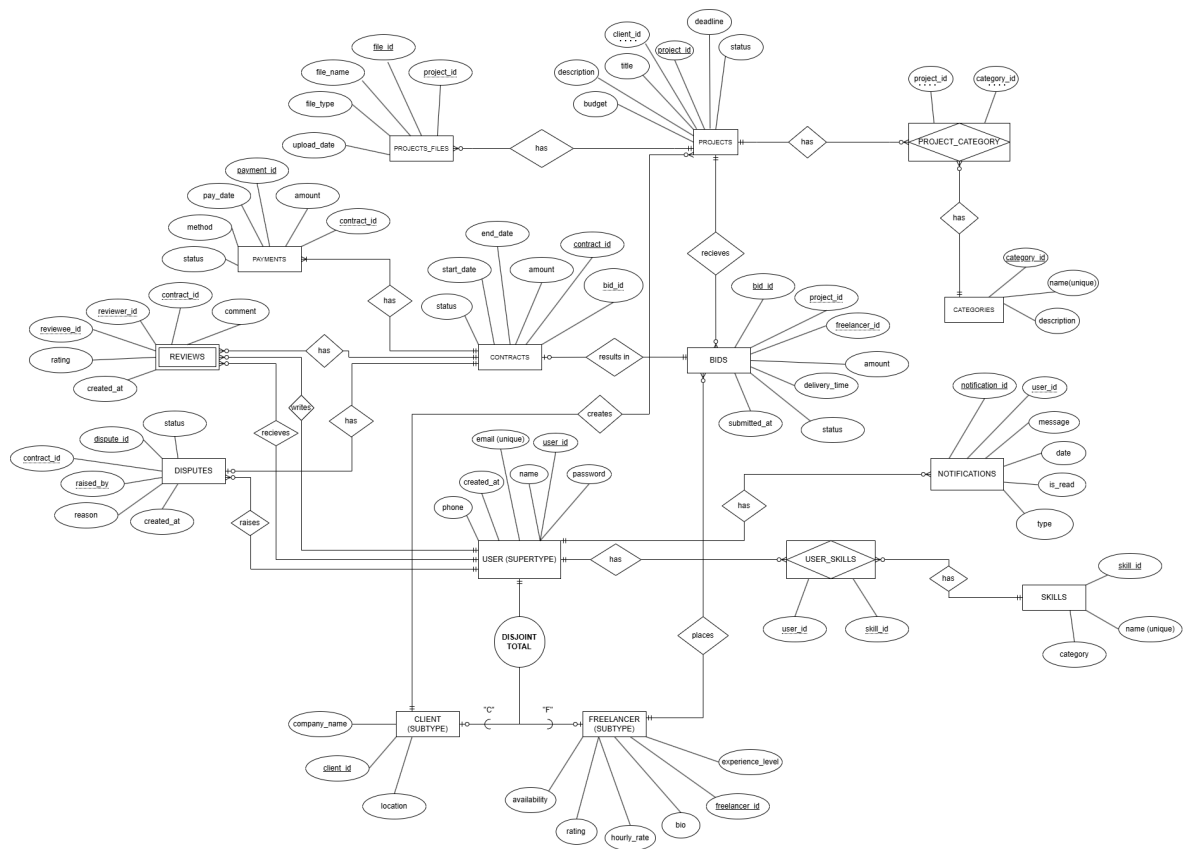


Figure 3.1: SkillLink Entity-Relationship Diagram

Chapter 4

Relational Data Model

4.1 Overview

The Relational Data Model (RDM) is the result of converting the ER diagram into actual table definitions. Every entity in the ER diagram becomes a table. Every attribute becomes a column. Every relationship becomes a foreign key. Every M:N relationship becomes a junction table with a composite primary key.

The RDM is what gets implemented directly in Oracle SQL. It defines exactly what tables exist, what columns each table has, what data type each column uses, and what constraints apply. In simple terms, the RDM is the blueprint that the CREATE TABLE statements are written from.

4.2 Conversion Rules Applied

The following rules were used to convert the ER diagram into the relational model:

- Each strong entity becomes a table with its own primary key
- Each subtype in a specialization becomes a separate table that shares the primary key of the supertype as both its primary key and foreign key
- Each weak entity becomes a table with a foreign key referencing its identifying entity, plus a surrogate primary key for practical use in Oracle
- Each M:N relationship becomes a junction table with a composite primary key made up of the two foreign keys

- Each 1:M relationship is represented by placing the foreign key on the many side
- Each 1:1 relationship is represented by placing the foreign key on the dependent side

4.3 Entity Classification

Before listing the tables, the classification of each entity is shown in Table 4.1.

Table 4.1: Entity Classification

Table	Type	Reason
users	Supertype	Base entity for all users in the system
client_profile	Subtype	Specialization of users, shares PK
freelancer_profile	Subtype	Specialization of users, shares PK
skills	Strong	Exists independently
categories	Strong	Exists independently
projects	Strong	Exists independently, owned by client
user_skills	Associative	Resolves M:N between users and skills
project_category	Associative	Resolves M:N between projects and categories
bids	Weak	Depends on projects and freelancer_profile
contracts	Strong	Created from an accepted bid
payments	Strong	Linked to a contract
reviews	Weak	Depends on contracts
project_files	Strong	Linked to a project
disputes	Strong	Linked to a contract
notifications	Strong	Linked to a user

4.4 All 15 Tables

4.4.1 users

The supertype table. Every person using the system has a record here regardless of whether they are a client or freelancer.

Table 4.2: users table

Column	Type	Constraint	Description
user_id	NUMBER	PK	Unique identifier, generated by sequence
name	VARCHAR(100)	NOT NULL	Full name of the user
email	VARCHAR(100)	UNIQUE, NOT NULL	Login email, must be unique
password	VARCHAR(100)	NOT NULL	Stored password
phone	VARCHAR(20)		Contact number, optional
created_at	DATE	DEFAULT SYSDATE	Account creation timestamp

4.4.2 client_profile

Subtype of users. Only users who are clients have a record here. The `client_id` is both the primary key and a foreign key referencing `users`.

Table 4.3: client_profile table

Column	Type	Constraint	Description
client_id	NUMBER	PK, FK	References user_id in users
company_name	VARCHAR(100)		Name of the client company, optional
location	VARCHAR(100)		City or region of the client

4.4.3 freelancer_profile

Subtype of users. Only users who are freelancers have a record here. The `freelancer_id` is both the primary key and a foreign key referencing `users`.

4.4.4 skills

Stores all skills that can be assigned to users.

4.4.5 user_skills

Associative table resolving the M:N relationship between users and skills. The composite primary key (`user_id`, `skill_id`) prevents duplicate entries.

4.4.6 categories

Stores project categories used to classify projects by domain.

Table 4.4: freelancer_profile table

Column	Type	Constraint	Description
freelancer_id	NUMBER	PK, FK	References user_id in users
bio	VARCHAR(500)	NOT NULL	Short professional description
hourly_rate	NUMBER(10,2)		Rate charged per hour
rating	NUMBER(4,2)	DEFAULT 0	Average rating from reviews
experience_level	VARCHAR(20)		beginner, intermediate, or expert
availability	VARCHAR(5)		yes or no

Table 4.5: skills table

Column	Type	Constraint	Description
skill_id	NUMBER	PK	Unique identifier
name	VARCHAR(100)	UNIQUE, NOT NULL	Name of the skill
category	VARCHAR(100)		Broad category such as Tech, Design, Writing

4.4.7 projects

Stores all projects posted by clients.

4.4.8 project_category

Associative table resolving the M:N relationship between projects and categories.

4.4.9 project_files

Stores metadata of files attached to projects.

4.4.10 bids

Weak entity. Stores bids submitted by freelancers on projects. A freelancer cannot bid twice on the same project due to the UNIQUE constraint on (project_id, freelancer_id).

4.4.11 contracts

Created when a bid is accepted. Linked to the winning bid through bid_id. The client and freelancer are reachable by joining through the bids table, keeping the design free of redundant foreign keys.

Table 4.6: user_skills table

Column	Type	Constraint	Description
user_id	NUMBER	PK, FK	References user_id in users
skill_id	NUMBER	PK, FK	References skill_id in skills

Table 4.7: categories table

Column	Type	Constraint	Description
category_id	NUMBER	PK	Unique identifier
name	VARCHAR(100)	UNIQUE, NOT NULL	Category name
description	VARCHAR(300)		Brief description of the category

4.4.12 payments

Stores payment transactions linked to contracts. One contract can have multiple payments for milestone-based work.

4.4.13 reviews

Weak entity. Stores reviews submitted after contract completion. Both the client and freelancer can leave one review each, referencing the same contract.

4.4.14 disputes

Stores disputes raised against contracts by either the client or freelancer.

4.4.15 notifications

Stores system notifications sent to users.

4.5 Full RDM Schema

The complete relational schema in notation form is listed below. Primary keys are underlined and foreign keys are marked with FK.

Table 4.8: projects table

Column	Type	Constraint	Description
project_id	NUMBER	PK	Unique identifier
client_id	NUMBER	FK, NOT NULL	References client_profile
title	VARCHAR(200)	NOT NULL	Project title
description	VARCHAR(1000)		Full project description
budget	NUMBER(10,2)	NOT NULL	Maximum budget set by client
deadline	DATE		Expected completion date
status	VARCHAR(20)	DEFAULT open	open, in_progress, completed, cancelled

Table 4.9: project_category table

Column	Type	Constraint	Description
project_id	NUMBER	PK, FK	References projects
category_id	NUMBER	PK, FK	References categories

users(user_id, name, email, password, phone, created_at)

client_profile(client_id [FK], company_name, location)

freelancer_profile(freelancer_id [FK], bio, hourly_rate, rating, experience_level, availability)

skills(skill_id, name, category)

user_skills(user_id [FK], skill_id [FK])

categories(category_id, name, description)

projects(project_id, client_id [FK], title, description, budget, deadline, status)

project_category(project_id [FK], category_id [FK])

project_files(file_id, project_id [FK], file_name, file_type, upload_date)

bids(bid_id, project_id [FK], freelancer_id [FK], amount, delivery_time, status, submitted_at)

contracts(contract_id, bid_id [FK], amount, start_date, end_date, status)

payments(payment_id, contract_id [FK], amount, pay_date, method, status)

reviews(review_id, contract_id [FK], reviewer_id [FK], reviewee_id [FK], rating, comment, created_at)

disputes(dispute_id, contract_id [FK], raised_by [FK], reason, status, created_at)

notifications(notification_id, user_id [FK], message, date_sent, is_read, type)

Table 4.10: project_files table

Column	Type	Constraint	Description
file_id	NUMBER	PK	Unique identifier
project_id	NUMBER	FK, NOT NULL	References projects
file_name	VARCHAR(200)	NOT NULL	Name of the uploaded file
file_type	VARCHAR(50)		File extension such as pdf, docx, xlsx
upload_date	DATE	DEFAULT SYSDATE	Date the file was attached

Table 4.11: bids table

Column	Type	Constraint	Description
bid_id	NUMBER	PK	Surrogate primary key
project_id	NUMBER	FK, NOT NULL	References projects
freelancer_id	NUMBER	FK, NOT NULL	References freelancer_profile
amount	NUMBER(10,2)	NOT NULL	Bid amount proposed by freelancer
delivery_time	NUMBER		Proposed delivery in days
status	VARCHAR(20)	DEFAULT pending	pending, accepted, or rejected
submitted_at	DATE	DEFAULT SYSDATE	Timestamp of bid submission
—	—	UNIQUE(project_id, freelancer_id)	Prevents duplicate bids

Table 4.12: contracts table

Column	Type	Constraint	Description
contract_id	NUMBER	PK	Unique identifier
bid_id	NUMBER	FK, UNIQUE, NOT NULL	References the accepted bid
amount	NUMBER(10,2)	NOT NULL	Agreed contract amount
start_date	DATE	DEFAULT SYSDATE	Contract start date
end_date	DATE		Expected end date
status	VARCHAR(20)	DEFAULT active	active, completed, or disputed

Table 4.13: payments table

Column	Type	Constraint	Description
payment_id	NUMBER	PK	Unique identifier
contract_id	NUMBER	FK, NOT NULL	References contracts
amount	NUMBER(10,2)	NOT NULL	Payment amount
pay_date	DATE	DEFAULT SYSDATE	Date payment was made
method	VARCHAR(20)		bank, easypaisa, jazzcash, or card
status	VARCHAR(20)	DEFAULT pending	paid, pending, or failed

Table 4.14: reviews table

Column	Type	Constraint	Description
review_id	NUMBER	PK	Unique identifier
contract_id	NUMBER	FK, NOT NULL	References contracts
reviewer_id	NUMBER	FK, NOT NULL	User who wrote the review
reviewee_id	NUMBER	FK, NOT NULL	User who received the review
rating	NUMBER	NOT NULL	Score given
comment	VARCHAR(500)		Written feedback
created_at	DATE	DEFAULT SYSDATE	Submission timestamp

Table 4.15: disputes table

Column	Type	Constraint	Description
dispute_id	NUMBER	PK	Unique identifier
contract_id	NUMBER	FK, NOT NULL	References contracts
raised_by	NUMBER	FK, NOT NULL	References users
reason	VARCHAR(500)		Description of the dispute
status	VARCHAR(20)	DEFAULT open	open, resolved, or rejected
created_at	DATE	DEFAULT SYSDATE	Timestamp

Table 4.16: notifications table

Column	Type	Constraint	Description
notification_id	NUMBER	PK	Unique identifier
user_id	NUMBER	FK, NOT NULL	References users
message	VARCHAR(500)	NOT NULL	Notification text
date_sent	DATE	DEFAULT SYSDATE	When it was sent
is_read	VARCHAR(1)	DEFAULT N	Y if read, N if unread
type	VARCHAR(50)		bid_accepted, payment, dispute, etc.

Chapter 5

Normalization

5.1 Overview

Normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity. It works by applying a series of rules called normal forms. Each normal form builds on the previous one, and a database that satisfies a higher normal form is considered better structured than one that only satisfies a lower one.

The goal is not just to follow rules for the sake of it. In practice, normalization means that the same piece of information is stored in only one place. If a freelancer changes their email address, that change happens in exactly one row in exactly one table. Nothing else needs to be updated. Without normalization, the same email might be stored in five different places and updating it in one place while missing the others creates inconsistent data.

SkillLink is normalized to the Third Normal Form (3NF). This section explains what each normal form requires and shows how the SkillLink tables satisfy each one with examples from the actual schema.

5.2 First Normal Form (1NF)

5.2.1 Rule

A table is in First Normal Form if:

- Every column contains atomic values — meaning each cell holds a single value, not

a list or a set

- There are no repeating groups of columns
- Every row is uniquely identifiable by a primary key

5.2.2 Violation Example

Suppose we had designed the system without normalization and stored skills directly inside the user record like this:

Table 5.1: Before 1NF — skills stored as a list inside one column

user_id	name	skills
7	Fatima Zahra	Web Development, Python, React Native
8	Usman Malik	Graphic Design, UI UX Design

This violates 1NF because the skills column contains multiple values in a single cell. You cannot query for all users who know Python without parsing the text, which is unreliable and inefficient.

5.2.3 How SkillLink Satisfies 1NF

Every column in every SkillLink table holds exactly one value. Skills are stored in a separate `skills` table and linked to users through the `user_skills` junction table. Each row in `user_skills` stores exactly one user-skill pair.

Table 5.2: After 1NF — user_skills junction table

user_id	skill_id
7	1
7	3
7	9
8	2
8	8

Every table has a clearly defined primary key. Every cell holds a single atomic value. All 15 tables in SkillLink satisfy 1NF.

5.3 Second Normal Form (2NF)

5.3.1 Rule

A table is in Second Normal Form if:

- It is already in 1NF
- Every non-key column is fully dependent on the entire primary key, not just part of it

Partial dependency only applies to tables with a composite primary key. If a non-key column depends on only one part of the composite key rather than both parts together, that is a partial dependency and it must be removed.

5.3.2 Violation Example

Suppose bids were designed as a single flat table like this:

Table 5.3: Before 2NF — partial dependency on `project_id` only

<code>project_id</code>	<code>freelancer_id</code>	<code>amount</code>	<code>project_title</code>	<code>budget</code>
1	7	45000	Portfolio Website	50000
1	9	48000	Portfolio Website	50000
2	8	12000	Logo Package	15000

Here the composite key is (`project_id`, `freelancer_id`). But `project_title` and `budget` depend only on `project_id`, not on the full composite key. This is a partial dependency. If the project title changes, it must be updated in every bid row for that project.

5.3.3 How SkillLink Satisfies 2NF

Project information is stored only in the `projects` table. The `bids` table stores only information that genuinely depends on the combination of project and freelancer together, which is the bid amount, delivery time, status, and submission timestamp.

Table 5.4: After 2NF — bids table contains only bid-specific data

<code>bid_id</code>	<code>project_id</code>	<code>freelancer_id</code>	<code>amount</code>	<code>delivery_time</code>	<code>status</code>
1	1	7	45000	25	accepted
2	1	9	48000	30	rejected
3	2	8	12000	10	accepted

The same principle applies to `user_skills` and `project_category`. Both junction

tables contain only foreign keys forming the composite primary key with no additional non-key columns that could create partial dependencies. All 15 tables in SkillLink satisfy 2NF.

5.4 Third Normal Form (3NF)

5.4.1 Rule

A table is in Third Normal Form if:

- It is already in 2NF
- No non-key column depends on another non-key column. In other words, there are no transitive dependencies

A transitive dependency means column C depends on column B, and column B depends on the primary key A. So C is only indirectly dependent on the primary key through B. This must be removed.

5.4.2 Violation Example

Suppose the freelancer profile was designed to include the city and the country of the freelancer like this:

Table 5.5: Before 3NF — transitive dependency between city and country

freelancer_id	name	hourly_rate	city	country
7	Fatima Zahra	2500	Lahore	Pakistan
8	Usman Malik	1500	Karachi	Pakistan
9	Zara Noor	2000	New York	USA

Here **country** depends on **city**, not directly on **freelancer_id**. If Lahore is always in Pakistan, that fact is repeated in every row for every freelancer from Lahore. This is a transitive dependency. If the country information for a city ever needed to change, every row would need updating.

5.4.3 How SkillLink Satisfies 3NF

SkillLink avoids transitive dependencies by separating concerns into distinct tables. The clearest example is the separation of user information from profile information.

Instead of storing all user and freelancer attributes in one flat table, SkillLink uses:

- **users** — stores only identity attributes: name, email, password, phone, created_at
- **freelancer_profile** — stores only freelancer-specific attributes: bio, hourly_rate, rating, experience_level, availability
- **client_profile** — stores only client-specific attributes: company_name, location

Each non-key column in every table depends directly and only on the primary key of that table. There is no column in any table that depends on another non-key column. All 15 tables in SkillLink satisfy 3NF.

Summary: All 15 tables in SkillLink satisfy 1NF, 2NF, and 3NF. Data redundancy is eliminated by storing each fact in exactly one place and linking tables through foreign keys rather than duplicating information across tables.

Chapter 6

Physical Implementation

6.1 Overview

The physical implementation is where the relational data model gets translated into actual Oracle SQL. This chapter covers the complete DDL (Data Definition Language) scripts used to build the SkillLink database from scratch. It includes the sequences for auto-generating IDs and all 15 CREATE TABLE statements written in the correct dependency order.

All scripts were written and executed in DBeaver Community Edition connected to Oracle XE 21c. The schema used is **FREELANCE**.

6.2 Table Creation Order

Oracle enforces foreign key constraints immediately at the time of table creation. This means a child table that references a parent table cannot be created before the parent table exists. The correct creation order for all 15 tables is shown below.

6.3 Sequences

Oracle does not have an `AUTO_INCREMENT` feature like MySQL. Sequences are used instead to generate unique numeric IDs automatically. One sequence is created for each table that has a surrogate primary key.

Listing 6.1: Sequences

```
-- one sequence per table with a surrogate primary key
```

Table 6.1: Table Creation Order

Step	Table	Reason
1	users	No dependencies, base table for all users
2	client_profile	References users
3	freelancer_profile	References users
4	skills	No dependencies
5	user_skills	References users and skills
6	categories	No dependencies
7	projects	References client_profile
8	project_category	References projects and categories
9	project_files	References projects
10	bids	References projects and freelancer_profile
11	contracts	References bids
12	payments	References contracts
13	reviews	References contracts and users
14	disputes	References contracts and users
15	notifications	References users

```

create sequence seq_users      start with 1 increment by 1;
create sequence seq_skills    start with 1 increment by 1;
create sequence seq_categories start with 1 increment by 1;
create sequence seq_projects  start with 1 increment by 1;
create sequence seq_files     start with 1 increment by 1;
create sequence seq_bids      start with 1 increment by 1;
create sequence seq_contracts  start with 1 increment by 1;
create sequence seq_payments  start with 1 increment by 1;
create sequence seq_reviews    start with 1 increment by 1;
create sequence seq_disputes   start with 1 increment by 1;
create sequence seq_notifications start with 1 increment by 1;

```

When inserting a row, the sequence is called using `seq_users.nextval` which returns the next available number automatically.

6.4 CREATE TABLE Statements

6.4.1 users

Listing 6.2: users

```
create table users (  
  user_id      number          primary key,  
  name         varchar(100)    not null,  
  email        varchar(100)    unique not null,  
  password     varchar(100)    not null,  
  phone        varchar(20),  
  created_at   date            default sysdate  
);
```

6.4.2 client_profile

Listing 6.3: client_profile

```
create table client_profile (  
  client_id    number          primary key,  
  company_name varchar(100),  
  location     varchar(100),  
  constraint fk_client foreign key (client_id)  
    references users(user_id)  
);
```

6.4.3 freelancer_profile

Listing 6.4: freelancer_profile

```
create table freelancer_profile (  
  freelancer_id number          primary key,  
  bio           varchar(500)    not null,  
  hourly_rate   number(10,2),  
  rating        number(4,2)     default 0,  
  experience_level varchar(20),  
  availability   varchar(5),  
  constraint fk_freelancer foreign key (freelancer_id)  
    references users(user_id)  
);
```

6.4.4 skills

Listing 6.5: skills

```
create table skills (  
  skill_id number          primary key,  
  name     varchar(100)    unique not null,  
  category varchar(100)  
);
```

6.4.5 user_skills

Listing 6.6: user_skills

```
create table user_skills (  
  user_id  number,  
  skill_id number,  
  primary key (user_id, skill_id),  
  constraint fk_us_user  foreign key (user_id)  
    references users(user_id),  
  constraint fk_us_skill foreign key (skill_id)  
    references skills(skill_id)  
);
```

6.4.6 categories

Listing 6.7: categories

```
create table categories (  
  category_id number      primary key,  
  name        varchar(100) unique not null,  
  description varchar(300)  
);
```

6.4.7 projects

Listing 6.8: projects

```
create table projects (  
  project_id  number      primary key,  
  client_id   number      not null,  
  title       varchar(200) not null,  
  description  varchar(1000),  
  budget      number(10,2) not null,  
  deadline    date,  
  status      varchar(20)  default 'open',  
  constraint fk_proj_client foreign key (client_id)  
    references client_profile(client_id)  
);
```

6.4.8 project_category

Listing 6.9: project_category

```
create table project_category (  
  project_id  number,  
  category_id number,  
  primary key (project_id, category_id),
```

```
constraint fk_pc_proj foreign key (project_id)
references projects(project_id),
constraint fk_pc_cat foreign key (category_id)
references categories(category_id)
);
```

6.4.9 project_files

Listing 6.10: project_files

```
create table project_files (
file_id          number          primary key,
project_id       number          not null,
file_name        varchar(200)    not null,
file_type        varchar(50),
upload_date      date            default sysdate,
constraint fk_pf_proj foreign key (project_id)
references projects(project_id)
);
```

6.4.10 bids

Listing 6.11: bids

```
create table bids (
bid_id           number          primary key,
project_id       number          not null,
freelancer_id    number          not null,
amount           number(10,2)    not null,
delivery_time    number,
status           varchar(20)     default 'pending',
submitted_at     date            default sysdate,
constraint fk_bid_proj foreign key (project_id)
references projects(project_id),
constraint fk_bid_free foreign key (freelancer_id)
references freelancer_profile(freelancer_id),
constraint uq_bid unique (project_id, freelancer_id)
);
```

6.4.11 contracts

Listing 6.12: contracts

```
create table contracts (
contract_id      number          primary key,
bid_id           number          unique not null,
amount           number(10,2)    not null,
```

```
start_date    date                default sysdate ,
end_date      date ,
status        varchar(20)         default 'active' ,
constraint fk_con_bid foreign key (bid_id)
              references bids(bid_id)
);
```

6.4.12 payments

Listing 6.13: payments

```
create table payments (
  payment_id   number              primary key ,
  contract_id  number              not null ,
  amount       number(10,2)        not null ,
  pay_date     date                default sysdate ,
  method       varchar(20) ,
  status       varchar(20)         default 'pending' ,
  constraint fk_pay_con foreign key (contract_id)
              references contracts(contract_id)
);
```

6.4.13 reviews

Listing 6.14: reviews

```
create table reviews (
  review_id    number              primary key ,
  contract_id  number              not null ,
  reviewer_id  number              not null ,
  reviewee_id  number              not null ,
  rating       number              not null ,
  comment      varchar(500) ,
  created_at   date                default sysdate ,
  constraint fk_rev_con      foreign key (contract_id)
              references contracts(contract_id) ,
  constraint fk_rev_reviewer foreign key (reviewer_id)
              references users(user_id) ,
  constraint fk_rev_reviewee foreign key (reviewee_id)
              references users(user_id)
);
```

6.4.14 disputes

Listing 6.15: disputes

```
create table disputes (
```



```
dispute_id    number           primary key,
contract_id   number           not null,
raised_by     number           not null,
reason        varchar(500),
status        varchar(20)      default 'open',
created_at    date             default sysdate,
constraint fk_dis_con foreign key (contract_id)
references contracts(contract_id),
constraint fk_dis_user foreign key (raised_by)
references users(user_id)
);
```

6.4.15 notifications

Listing 6.16: notifications

```
create table notifications (
  notification_id number           primary key,
  user_id         number           not null,
  message         varchar(500)     not null,
  date_sent       date             default sysdate,
  is_read         varchar(1)       default 'N',
  type            varchar(50),
  constraint fk_notif_user foreign key (user_id)
references users(user_id)
);
```

6.5 Cleanup Script

To drop all tables and sequences and rebuild from scratch, the following cleanup script is run first. Tables must be dropped in reverse dependency order so that child tables are removed before parent tables.

Listing 6.17: Cleanup – Drop All

```
-- drop tables in reverse dependency order
drop table notifications;
drop table disputes;
drop table reviews;
drop table payments;
drop table contracts;
drop table bids;
drop table project_files;
drop table project_category;
```

```
drop table projects;
drop table categories;
drop table user_skills;
drop table skills;
drop table freelancer_profile;
drop table client_profile;
drop table users;

-- drop all sequences
drop sequence seq_users;
drop sequence seq_skills;
drop sequence seq_categories;
drop sequence seq_projects;
drop sequence seq_files;
drop sequence seq_bids;
drop sequence seq_contracts;
drop sequence seq_payments;
drop sequence seq_reviews;
drop sequence seq_disputes;
drop sequence seq_notifications;
```

Note: Running the cleanup script will permanently delete all tables and data. It is used only when rebuilding the database from scratch during development.

Chapter 7

Data Insertion

7.1 Overview

After all 15 tables were created, realistic sample data was inserted into every table. The data follows the correct insertion order — parent tables first, child tables after — to satisfy all foreign key constraints. All IDs are generated using the Oracle sequences defined in Chapter 6.

The sample data models a realistic freelance marketplace scenario with Pakistani and international users, real-world project types, and a complete workflow from project posting through bidding, contracting, payment, and review.

7.2 Insertion Order

Data must be inserted in the same dependency order as table creation. Inserting a bid before the project and freelancer exist would violate the foreign key constraint and Oracle would reject the insert.

7.3 Sample Data

7.3.1 users

Listing 7.1: Insert users

```
insert into users values (seq_users.nextval, 'Zairish Fatima', '
↳ zairish@email.com', 'pass123', '0300111111', sysdate);
```

Table 7.1: Data Insertion Order

Step	Table	Reason
1	users	No dependencies
2	client_profile	Depends on users
3	freelancer_profile	Depends on users
4	skills	No dependencies
5	user_skills	Depends on users and skills
6	categories	No dependencies
7	projects	Depends on client_profile
8	project_category	Depends on projects and categories
9	project_files	Depends on projects
10	bids	Depends on projects and freelancer_profile
11	contracts	Depends on bids
12	payments	Depends on contracts
13	reviews	Depends on contracts and users
14	disputes	Depends on contracts and users
15	notifications	Depends on users

```

insert into users values (seq_users.nextval, 'Maryam Qadir', '
↳ maryam@email.com', 'pass123', '0300222222', sysdate);
insert into users values (seq_users.nextval, 'Saim Zafar', 'saim@email
↳ .com', 'pass123', '0300333333', sysdate);
insert into users values (seq_users.nextval, 'Ali Raza', 'ali@email.
↳ com', 'pass123', '0300444444', sysdate);
insert into users values (seq_users.nextval, 'Fatima Zahra', '
↳ fatima@email.com', 'pass123', '0300555555', sysdate);
insert into users values (seq_users.nextval, 'Usman Malik', '
↳ usman@email.com', 'pass123', '0300666666', sysdate);
insert into users values (seq_users.nextval, 'Zara Noor', 'zara@email.
↳ com', 'pass123', '0300777777', sysdate);
insert into users values (seq_users.nextval, 'Hassan Ahmed', '
↳ hassan@email.com', 'pass123', '0300888888', sysdate);
insert into users values (seq_users.nextval, 'Sana Khan', 'sana@email.
↳ com', 'pass123', '0300999999', sysdate);
insert into users values (seq_users.nextval, 'Bilal Tariq', '
↳ bilal@email.com', 'pass123', '0301010101', sysdate);
commit;

```

7.3.2 client_profile

Listing 7.2: Insert client_profile

```
insert into client_profile values (1, 'TechNova Solutions', 'Islamabad'
↳ ');
insert into client_profile values (2, 'Creative Hub', 'Lahore');
insert into client_profile values (3, 'DigitalEdge', 'Karachi');
insert into client_profile values (4, 'Startup Ventures', 'Rawalpindi'
↳ );
insert into client_profile values (5, 'WebCraft Co', 'Islamabad');
commit;
```

7.3.3 freelancer_profile

Listing 7.3: Insert freelancer_profile

```
insert into freelancer_profile values (6, 'Full stack developer with 4
↳ years experience', 2500, 4.7, 'expert', 'yes');
insert into freelancer_profile values (7, 'UI/UX designer specializing
↳ in mobile apps', 2000, 4.5, 'intermediate', 'yes');
insert into freelancer_profile values (8, 'Graphic designer and brand
↳ identity expert', 1500, 4.2, 'intermediate', 'yes');
insert into freelancer_profile values (9, 'Backend developer
↳ specializing in Node.js', 3000, 4.8, 'expert', 'no');
insert into freelancer_profile values (10, 'Content writer and SEO
↳ specialist', 1000, 4.0, 'beginner', 'yes');
commit;
```

7.3.4 skills

Listing 7.4: Insert skills

```
insert into skills values (seq_skills.nextval, 'Web Development', '
↳ Tech');
insert into skills values (seq_skills.nextval, 'Graphic Design', '
↳ Design');
insert into skills values (seq_skills.nextval, 'React Native', 'Tech')
↳ ;
insert into skills values (seq_skills.nextval, 'UI UX Design', 'Design
↳ ');
insert into skills values (seq_skills.nextval, 'Node.js', 'Tech');
insert into skills values (seq_skills.nextval, 'Content Writing', '
↳ Writing');
insert into skills values (seq_skills.nextval, 'SEO', 'Marketing');
insert into skills values (seq_skills.nextval, 'Brand Identity', '
↳ Design');
insert into skills values (seq_skills.nextval, 'Python', 'Tech');
insert into skills values (seq_skills.nextval, 'Database Design', '
↳ Tech');
```

```
commit;
```

7.3.5 user_skills

Listing 7.5: Insert user_skills

```
insert into user_skills values (6, 1);
insert into user_skills values (6, 5);
insert into user_skills values (6, 10);
insert into user_skills values (7, 4);
insert into user_skills values (7, 3);
insert into user_skills values (8, 2);
insert into user_skills values (8, 8);
insert into user_skills values (9, 5);
insert into user_skills values (9, 9);
insert into user_skills values (10, 6);
insert into user_skills values (10, 7);
commit;
```

7.3.6 categories

Listing 7.6: Insert categories

```
insert into categories values (seq_categories.nextval, 'Web
↳ Development', 'Projects related to websites and web apps');
insert into categories values (seq_categories.nextval, 'Mobile
↳ Development', 'Android and iOS application projects');
insert into categories values (seq_categories.nextval, 'Graphic Design
↳ ', 'Logo, branding, and visual design projects');
insert into categories values (seq_categories.nextval, 'Content
↳ Writing', 'Blog posts, articles, and copywriting');
insert into categories values (seq_categories.nextval, 'Database', '
↳ Database design and administration projects');
commit;
```

7.3.7 projects

Listing 7.7: Insert projects

```
insert into projects values (seq_projects.nextval, 1, 'Portfolio
↳ Website', 'Build a personal portfolio website with React', 50000,
↳ to_date('2026-06-01','YYYY-MM-DD'), 'open');
insert into projects values (seq_projects.nextval, 2, 'Logo Design
↳ Package', 'Design a complete brand identity package', 15000,
↳ to_date('2026-05-15','YYYY-MM-DD'), 'open');
insert into projects values (seq_projects.nextval, 3, 'E-Commerce
↳ Backend', 'Node.js REST API for an online store', 80000, to_date('
↳ 2026-07-01','YYYY-MM-DD'), 'in_progress');
```

```
insert into projects values (seq_projects.nextval, 4, 'Mobile App UI',
↳ 'Design UI screens for a fitness tracking app', 35000, to_date('
↳ 2026-06-15', 'YYYY-MM-DD'), 'open');
insert into projects values (seq_projects.nextval, 5, 'Blog Content',
↳ 'Write 10 SEO-optimized blog posts', 12000, to_date('2026-05-30', '
↳ YYYY-MM-DD'), 'completed');
commit;
```

7.3.8 project_category

Listing 7.8: Insert project_category

```
insert into project_category values (1, 1);
insert into project_category values (2, 3);
insert into project_category values (3, 1);
insert into project_category values (3, 5);
insert into project_category values (4, 2);
insert into project_category values (5, 4);
commit;
```

7.3.9 project_files

Listing 7.9: Insert project_files

```
insert into project_files values (seq_files.nextval, 1, '
↳ portfolio_requirements.pdf', 'pdf', sysdate);
insert into project_files values (seq_files.nextval, 2, 'brand_brief.
↳ docx', 'docx', sysdate);
insert into project_files values (seq_files.nextval, 3, '
↳ api_specification.pdf', 'pdf', sysdate);
insert into project_files values (seq_files.nextval, 4, '
↳ app_wireframes.fig', 'fig', sysdate);
insert into project_files values (seq_files.nextval, 5, '
↳ content_topics.xlsx', 'xlsx', sysdate);
commit;
```

7.3.10 bids

Listing 7.10: Insert bids

```
insert into bids values (seq_bids.nextval, 1, 6, 45000, 25, 'accepted'
↳ , sysdate);
insert into bids values (seq_bids.nextval, 1, 7, 48000, 30, 'rejected'
↳ , sysdate);
insert into bids values (seq_bids.nextval, 2, 8, 12000, 10, 'accepted'
↳ , sysdate);
insert into bids values (seq_bids.nextval, 3, 9, 75000, 45, 'accepted'
↳ , sysdate);
```

```
insert into bids values (seq_bids.nextval, 4, 7, 32000, 20, 'pending',
↳ sysdate);
insert into bids values (seq_bids.nextval, 5, 10, 11000, 14, 'accepted
↳ ', sysdate);
insert into bids values (seq_bids.nextval, 3, 6, 78000, 40, 'rejected'
↳ , sysdate);
commit;
```

7.3.11 contracts

Listing 7.11: Insert contracts

```
insert into contracts values (seq_contracts.nextval, 1, 45000, sysdate
↳ , to_date('2026-06-01','YYYY-MM-DD'), 'active');
insert into contracts values (seq_contracts.nextval, 3, 12000, sysdate
↳ , to_date('2026-05-15','YYYY-MM-DD'), 'completed');
insert into contracts values (seq_contracts.nextval, 4, 75000, sysdate
↳ , to_date('2026-07-01','YYYY-MM-DD'), 'active');
insert into contracts values (seq_contracts.nextval, 6, 11000, sysdate
↳ , to_date('2026-05-30','YYYY-MM-DD'), 'completed');
commit;
```

7.3.12 payments

Listing 7.12: Insert payments

```
insert into payments values (seq_payments.nextval, 1, 22500, sysdate,
↳ 'bank', 'paid');
insert into payments values (seq_payments.nextval, 2, 12000, sysdate,
↳ 'easypaisa', 'paid');
insert into payments values (seq_payments.nextval, 3, 37500, sysdate,
↳ 'bank', 'paid');
insert into payments values (seq_payments.nextval, 4, 11000, sysdate,
↳ 'jazzcash', 'paid');
insert into payments values (seq_payments.nextval, 1, 22500, sysdate,
↳ 'bank', 'pending');
commit;
```

7.3.13 reviews

Listing 7.13: Insert reviews

```
insert into reviews values (seq_reviews.nextval, 2, 2, 8, 5, '
↳ Excellent logo work, very professional', sysdate);
insert into reviews values (seq_reviews.nextval, 2, 8, 2, 4, 'Client
↳ was clear about requirements', sysdate);
insert into reviews values (seq_reviews.nextval, 4, 5, 10, 4, 'Good
↳ content, delivered on time', sysdate);
```



```
insert into reviews values (seq_reviews.nextval, 4, 10, 5, 5, 'Great
↳ client, very responsive', sysdate);
commit;
```

7.3.14 disputes

Listing 7.14: Insert disputes

```
insert into disputes values (seq_disputes.nextval, 3, 4, 'Freelancer
↳ missed two milestones without notice', 'open', sysdate);
commit;
```

7.3.15 notifications

Listing 7.15: Insert notifications

```
insert into notifications values (seq_notifications.nextval, 6, 'Your
↳ bid on Portfolio Website has been accepted', sysdate, 'N', '
↳ bid_accepted');
insert into notifications values (seq_notifications.nextval, 8, 'Your
↳ bid on Logo Design Package has been accepted', sysdate, 'N', '
↳ bid_accepted');
insert into notifications values (seq_notifications.nextval, 9, 'Your
↳ bid on E-Commerce Backend has been accepted', sysdate, 'N', '
↳ bid_accepted');
insert into notifications values (seq_notifications.nextval, 1, '
↳ Payment of 22500 received for Portfolio Website', sysdate, 'Y', '
↳ payment');
insert into notifications values (seq_notifications.nextval, 4, 'A
↳ dispute has been raised on your contract', sysdate, 'N', 'dispute')
↳ ;
commit;
```

Note: All INSERT statements use `seq_name.nextval` for primary keys. The user IDs for client and freelancer profiles are entered manually (1–5 for clients, 6–10 for freelancers) because they reference existing `user_id` values rather than generating new ones.

Chapter 8

SQL Queries

8.1 Overview

This chapter presents 15 SQL queries written against the SkillLink database. The queries demonstrate a range of SQL techniques including INNER JOIN, LEFT JOIN, aggregate functions, GROUP BY, HAVING, subqueries, and CASE expressions. Each query is followed by an explanation of what it retrieves and why it is useful in the context of the system.

8.2 Query 1 — All Projects with Client Name

Retrieves every project along with the name of the client who posted it. Uses INNER JOIN between projects, client_profile, and users.

Listing 8.1: Query 1: All projects with client name

```
SELECT p.project_id, p.title, p.budget, p.status,
       u.name AS client_name
FROM projects p
INNER JOIN client_profile cp ON p.client_id = cp.client_id
INNER JOIN users u ON cp.client_id = u.user_id
ORDER BY p.project_id;
```

8.3 Query 2 — All Bids with Freelancer Name and Project Title

Retrieves every bid along with the freelancer who submitted it and the project it was placed on.

Listing 8.2: Query 2: All bids with freelancer and project

```
SELECT b.bid_id, p.title AS project_title,
       u.name AS freelancer_name,
       b.amount, b.status
FROM bids b
INNER JOIN projects p ON b.project_id = p.project_id
INNER JOIN freelancer_profile fp ON b.freelancer_id = fp.freelancer_id
INNER JOIN users u ON fp.freelancer_id = u.user_id
ORDER BY b.bid_id;
```

8.4 Query 3 — Active Contracts with Client and Freelancer Names

Retrieves all active contracts and shows both the client and freelancer involved by joining through bids.

Listing 8.3: Query 3: Active contracts with both parties

```
SELECT c.contract_id, c.amount, c.status,
       uc.name AS client_name,
       uf.name AS freelancer_name
FROM contracts c
INNER JOIN bids b ON c.bid_id = b.bid_id
INNER JOIN client_profile cp ON b.project_id IN (
    SELECT project_id FROM projects WHERE client_id = cp.client_id)
INNER JOIN projects pr ON b.project_id = pr.project_id
INNER JOIN client_profile cp2 ON pr.client_id = cp2.client_id
INNER JOIN users uc ON cp2.client_id = uc.user_id
INNER JOIN freelancer_profile fp ON b.freelancer_id = fp.freelancer_id
INNER JOIN users uf ON fp.freelancer_id = uf.user_id
WHERE c.status = 'active';
```

8.5 Query 4 — Total Payments Received per Freelancer

Uses SUM and GROUP BY to calculate the total amount paid to each freelancer across all their contracts.

Listing 8.4: Query 4: Total payments per freelancer

```
SELECT u.name AS freelancer_name,
       SUM(pay.amount) AS total_earned
FROM payments pay
INNER JOIN contracts c ON pay.contract_id = c.contract_id
INNER JOIN bids b ON c.bid_id = b.bid_id
INNER JOIN freelancer_profile fp ON b.freelancer_id = fp.freelancer_id
INNER JOIN users u ON fp.freelancer_id = u.user_id
WHERE pay.status = 'paid'
GROUP BY u.name
ORDER BY total_earned DESC;
```

8.6 Query 5 — Number of Bids per Project

Counts how many bids each project has received using COUNT and GROUP BY.

Listing 8.5: Query 5: Bid count per project

```
SELECT p.title AS project_title,
       COUNT(b.bid_id) AS total_bids
FROM projects p
LEFT JOIN bids b ON p.project_id = b.project_id
GROUP BY p.title
ORDER BY total_bids DESC;
```

8.7 Query 6 — Top Rated Freelancers

Retrieves freelancers with a rating above 4.0, ordered from highest to lowest rating.

Listing 8.6: Query 6: Top rated freelancers

```
SELECT u.name AS freelancer_name,
       fp.rating, fp.experience_level,
       fp.hourly_rate
FROM freelancer_profile fp
INNER JOIN users u ON fp.freelancer_id = u.user_id
WHERE fp.rating > 4.0
ORDER BY fp.rating DESC;
```

8.8 Query 7 — Projects with No Bids (LEFT JOIN)

Uses LEFT JOIN to find projects that have not received any bids yet.

Listing 8.7: Query 7: Projects with no bids

```
SELECT p.project_id, p.title, p.budget, p.status
FROM projects p
LEFT JOIN bids b ON p.project_id = b.project_id
WHERE b.bid_id IS NULL;
```

8.9 Query 8 — Average Bid Amount per Project

Calculates the average bid amount submitted for each project using AVG and GROUP BY.

Listing 8.8: Query 8: Average bid amount per project

```
SELECT p.title AS project_title,
       COUNT(b.bid_id) AS num_bids,
       ROUND(AVG(b.amount), 2) AS avg_bid_amount,
       MIN(b.amount) AS lowest_bid,
       MAX(b.amount) AS highest_bid
FROM projects p
INNER JOIN bids b ON p.project_id = b.project_id
GROUP BY p.title
ORDER BY avg_bid_amount DESC;
```

8.10 Query 9 — All Skills of Each Freelancer

Joins freelancer_profile, user_skills, and skills to list every skill assigned to each freelancer.

Listing 8.9: Query 9: Skills per freelancer

```
SELECT u.name AS freelancer_name,
       s.name AS skill_name,
       s.category
FROM users u
INNER JOIN freelancer_profile fp ON u.user_id = fp.freelancer_id
INNER JOIN user_skills us ON u.user_id = us.user_id
```

```
INNER JOIN skills s ON us.skill_id = s.skill_id
ORDER BY u.name, s.category;
```

8.11 Query 10 — Clients Who Have Spent More Than 20000

Uses HAVING to filter clients whose total payment releases exceed 20000.

Listing 8.10: Query 10: High-spending clients

```
SELECT u.name AS client_name,
       SUM(pay.amount) AS total_spent
FROM payments pay
INNER JOIN contracts c ON pay.contract_id = c.contract_id
INNER JOIN bids b ON c.bid_id = b.bid_id
INNER JOIN projects pr ON b.project_id = pr.project_id
INNER JOIN client_profile cp ON pr.client_id = cp.client_id
INNER JOIN users u ON cp.client_id = u.user_id
WHERE pay.status = 'paid'
GROUP BY u.name
HAVING SUM(pay.amount) > 20000
ORDER BY total_spent DESC;
```

8.12 Query 11 — All Reviews with Reviewer and Reviewee Names

Joins reviews with users twice using aliases to show both the reviewer and reviewee name.

Listing 8.11: Query 11: Reviews with names

```
SELECT r.review_id,
       u1.name AS reviewer_name,
       u2.name AS reviewee_name,
       r.rating, r.comment
FROM reviews r
INNER JOIN users u1 ON r.reviewer_id = u1.user_id
INNER JOIN users u2 ON r.reviewee_id = u2.user_id
ORDER BY r.review_id;
```

8.13 Query 12 — Freelancers Who Have Never Won a Contract

Uses LEFT JOIN and IS NULL to find freelancers who submitted bids but never had one accepted.

Listing 8.12: Query 12: Freelancers with no contracts

```
SELECT u.name AS freelancer_name,
       fp.experience_level, fp.rating
FROM freelancer_profile fp
INNER JOIN users u ON fp.freelancer_id = u.user_id
LEFT JOIN bids b ON fp.freelancer_id = b.freelancer_id
      AND b.status = 'accepted'
WHERE b.bid_id IS NULL;
```

8.14 Query 13 — Payment Status Summary

Uses CASE and COUNT to produce a summary of how many payments are paid, pending, or failed.

Listing 8.13: Query 13: Payment status summary

```
SELECT
  COUNT(CASE WHEN status = 'paid' THEN 1 END) AS paid_count,
  COUNT(CASE WHEN status = 'pending' THEN 1 END) AS pending_count,
  COUNT(CASE WHEN status = 'failed' THEN 1 END) AS failed_count,
  SUM(CASE WHEN status = 'paid' THEN amount ELSE 0 END) AS
↪ total_paid
FROM payments;
```

8.15 Query 14 — Open Disputes with Contract and User Details

Retrieves all open disputes along with the contract they relate to and the user who raised them.

Listing 8.14: Query 14: Open disputes

```
SELECT d.dispute_id, d.reason, d.status,
       c.contract_id, c.amount AS contract_amount,
```

```
        u.name AS raised_by_name
FROM disputes d
INNER JOIN contracts c ON d.contract_id = c.contract_id
INNER JOIN users u ON d.raised_by = u.user_id
WHERE d.status = 'open'
ORDER BY d.created_at;
```

8.16 Query 15 — Full Project Summary

A comprehensive query joining 6 tables to produce a full summary of each project including client name, number of bids, accepted bid amount, and contract status.

Listing 8.15: Query 15: Full project summary

```
SELECT p.project_id, p.title,
       u.name AS client_name,
       p.budget, p.status AS project_status,
       COUNT(b.bid_id) AS total_bids,
       c.amount AS contract_amount,
       c.status AS contract_status
FROM projects p
INNER JOIN client_profile cp ON p.client_id = cp.client_id
INNER JOIN users u ON cp.client_id = u.user_id
LEFT JOIN bids b ON p.project_id = b.project_id
LEFT JOIN contracts c ON b.bid_id = c.bid_id
      AND b.status = 'accepted'
GROUP BY p.project_id, p.title, u.name,
         p.budget, p.status, c.amount, c.status
ORDER BY p.project_id;
```

Summary: All 15 queries run against the SkillLink Oracle XE database using the sample data inserted in Chapter 7. They demonstrate INNER JOIN, LEFT JOIN, aggregate functions (SUM, COUNT, AVG, MIN, MAX), GROUP BY, HAVING, CASE expressions, and multi-table joins across up to 6 tables in a single query.

Chapter 9

Backend Architecture

9.1 Overview

The SkillLink backend is a Node.js server built with the Express framework. It connects to Oracle XE 21c using the `oracledb` npm package and exposes 35 REST API endpoints across 10 route files. Every endpoint validates the user's JWT token before executing any query. The backend is the only layer that communicates directly with the Oracle database.

9.2 Server Setup

The entry point is `server.js`. It loads environment variables, creates the Oracle connection pool, registers all route files, and starts the HTTP server.

Listing 9.1: `server.js`

```
const express = require('express');
const oracledb = require('oracledb');
const cors = require('cors');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(express.json());

// initialize oracle connection pool
async function startServer() {
  await oracledb.createPool({
    user: process.env.DB_USER,
```

```
    password: process.env.DB_PASSWORD,
    connectionString: process.env.DB_CONNECT
  });

// register all route files
app.use('/api/auth', require('./routes/auth'));
app.use('/api/users', require('./routes/users'));
app.use('/api/projects', require('./routes/projects'));
app.use('/api/bids', require('./routes/bids'));
app.use('/api/contracts', require('./routes/contracts'));
app.use('/api/payments', require('./routes/payments'));
app.use('/api/reviews', require('./routes/reviews'));
app.use('/api/disputes', require('./routes/disputes'));
app.use('/api/notifications', require('./routes/notifications'));
app.use('/api/admin', require('./routes/admin'));

app.listen(process.env.PORT || 5000, () => {
  console.log('SkillLink server running on port 5000');
});
}

startServer();
```

9.3 Database Connection Helper

A shared helper function runs every SQL query. It gets a connection from the pool, executes the query, releases the connection, and returns the result.

Listing 9.2: db.js – connection helper

```
const oracledb = require('oracledb');

async function runQuery(sql, params = [], options = {}) {
  let connection;
  try {
    connection = await oracledb.getConnection();
    const result = await connection.execute(sql, params, {
      outFormat: oracledb.OUT_FORMAT_OBJECT,
      autoCommit: true,
      ...options
    });
    return result;
  } finally {
```

```
    if (connection) await connection.close();
  }
}

module.exports = { runQuery };
```

9.4 Authentication Middleware

Every protected route passes through the JWT middleware before reaching the handler. It verifies the token and attaches the decoded user object to the request.

Listing 9.3: middleware/auth.js

```
const jwt = require('jsonwebtoken');

function verifyToken(req, res, next) {
  const token = req.headers['authorization']?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token provided'
  ↪ });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    return res.status(403).json({ error: 'Invalid token' });
  }
}

function requireRole(role) {
  return (req, res, next) => {
    if (req.user.role !== role)
      return res.status(403).json({ error: 'Access denied' });
    next();
  };
}

module.exports = { verifyToken, requireRole };
```

9.5 Route File: auth.js

Handles user registration and login. Registration is a multi-table transaction that inserts into users and then into client_profile or freelancer_profile depending on the

role.

Listing 9.4: routes/auth.js – register and login

```
const express = require('express');
const router = express.Router();
const oracledb = require('oracledb');
const jwt = require('jsonwebtoken');

// POST /api/auth/register
router.post('/register', async (req, res) => {
  const { name, email, password, phone, role,
    company_name, location,
    bio, hourly_rate, experience_level } = req.body;
  let connection;
  try {
    connection = await oracledb.getConnection();

    // insert into users
    const userResult = await connection.execute(
      'INSERT INTO users VALUES
        (seq_users.nextval, :name, :email, :password, :phone, SYSDATE)
        RETURNING user_id INTO :id',
      { name, email, password, phone,
        id: { dir: oracledb.BIND_OUT, type: oracledb.NUMBER } },
      { autoCommit: false }
    );
    const userId = userResult.outBinds.id[0];

    // insert into subtype table based on role
    if (role === 'client') {
      await connection.execute(
        'INSERT INTO client_profile VALUES
          (:id, :company_name, :location)',
        { id: userId, company_name, location },
        { autoCommit: false }
      );
    } else {
      await connection.execute(
        'INSERT INTO freelancer_profile VALUES
          (:id, :bio, :hourly_rate, 0, :experience_level, 'yes')',
        { id: userId, bio, hourly_rate, experience_level },
        { autoCommit: false }
      );
    }
  }
}
```

```
    await connection.commit();
    res.json({ message: 'Registration successful', userId });
  } catch (err) {
    if (connection) await connection.rollback();
    res.status(500).json({ error: err.message });
  } finally {
    if (connection) await connection.close();
  }
});

// POST /api/auth/login
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const { runQuery } = require('../db');
  const result = await runQuery(
    'SELECT u.user_id, u.name, u.email,
           CASE WHEN cp.client_id IS NOT NULL THEN 'client'
                WHEN fp.freelancer_id IS NOT NULL THEN 'freelancer'
                ELSE 'admin' END AS role
    FROM users u
    LEFT JOIN client_profile cp ON u.user_id = cp.client_id
    LEFT JOIN freelancer_profile fp ON u.user_id = fp.freelancer_id
    WHERE u.email = :email AND u.password = :password',
    [email, password]
  );
  if (!result.rows.length)
    return res.status(401).json({ error: 'Invalid credentials' });
  const user = result.rows[0];
  const token = jwt.sign(
    { id: user.USER_ID, name: user.NAME, role: user.ROLE },
    process.env.JWT_SECRET,
    { expiresIn: '24h' }
  );
  res.json({ token, user });
});

module.exports = router;
```

9.6 Route File: projects.js

Full CRUD for projects plus a JOIN query to fetch projects with client names.

Listing 9.5: routes/projects.js – full CRUD

```

const express = require('express');
const router = express.Router();
const { runQuery } = require('../db');
const { verifyToken, requireRole } = require('../middleware/auth');

// GET /api/projects -- all projects with client name (JOIN)
router.get('/', verifyToken, async (req, res) => {
  try {
    const result = await runQuery(
      'SELECT p.project_id, p.title, p.description,
        p.budget, p.deadline, p.status,
        u.name AS client_name
      FROM projects p
      INNER JOIN client_profile cp ON p.client_id = cp.client_id
      INNER JOIN users u ON cp.client_id = u.user_id
      ORDER BY p.project_id DESC'
    );
    res.json(result.rows);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

// GET /api/projects/:id -- single project with full details
router.get('/:id', verifyToken, async (req, res) => {
  try {
    const result = await runQuery(
      'SELECT p.*, u.name AS client_name,
        COUNT(b.bid_id) AS total_bids
      FROM projects p
      INNER JOIN client_profile cp ON p.client_id = cp.client_id
      INNER JOIN users u ON cp.client_id = u.user_id
      LEFT JOIN bids b ON p.project_id = b.project_id
      WHERE p.project_id = :id
      GROUP BY p.project_id, p.client_id, p.title,
        p.description, p.budget, p.deadline,
        p.status, u.name'
    );
    [req.params.id]
  );
  if (!result.rows.length)
    return res.status(404).json({ error: 'Project not found' });
  res.json(result.rows[0]);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

// GET /api/projects/client/:clientId -- all projects by a client

```

```

router.get('/client/:clientId', verifyToken, async (req, res) => {
  try {
    const result = await runQuery(
      'SELECT p.project_id, p.title, p.budget,
        p.status, p.deadline,
        COUNT(b.bid_id) AS total_bids
      FROM projects p
      LEFT JOIN bids b ON p.project_id = b.project_id
      WHERE p.client_id = :id
      GROUP BY p.project_id, p.title, p.budget,
        p.status, p.deadline
      ORDER BY p.project_id DESC',
      [req.params.clientId]
    );
    res.json(result.rows);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

// POST /api/projects -- create project (client only)
router.post('/', verifyToken, requireRole('client'), async (req, res)
↳ => {
  const { title, description, budget, deadline } = req.body;
  try {
    await runQuery(
      'INSERT INTO projects VALUES
        (seq_projects.nextval, :client_id, :title,
         :description, :budget,
         TO_DATE(:deadline, 'YYYY-MM-DD'), 'open')',
      [req.user.id, title, description, budget, deadline]
    );
    res.json({ message: 'Project created successfully' });
  } catch (err) { res.status(500).json({ error: err.message }); }
});

// PUT /api/projects/:id -- update project
router.put('/:id', verifyToken, requireRole('client'), async (req, res
↳ ) => {
  const { title, description, budget, deadline, status } = req.body;
  try {
    await runQuery(
      'UPDATE projects SET
        title = :title,
        description = :description,
        budget = :budget,
        deadline = TO_DATE(:deadline, 'YYYY-MM-DD'),

```

```

        status = :status
        WHERE project_id = :id AND client_id = :client_id',
        [title, description, budget, deadline,
        status, req.params.id, req.user.id]
    );
    res.json({ message: 'Project updated successfully' });
} catch (err) { res.status(500).json({ error: err.message }); }
});

// DELETE /api/projects/:id -- delete project
router.delete('/:id', verifyToken, requireRole('client'),
    async (req, res) => {
    try {
        await runQuery(
            'DELETE FROM projects
            WHERE project_id = :id AND client_id = :client_id',
            [req.params.id, req.user.id]
        );
        res.json({ message: 'Project deleted successfully' });
    } catch (err) { res.status(500).json({ error: err.message }); }
});

module.exports = router;

```

9.7 Route File: bids.js

Full CRUD for bids. The accept bid endpoint is a transaction that updates the bid status and creates a contract atomically.

Listing 9.6: routes/bids.js – full CRUD with transaction

```

const express = require('express');
const router = express.Router();
const { runQuery } = require('../db');
const oracledb = require('oracledb');
const { verifyToken, requireRole } = require('../middleware/auth');

// GET /api/bids/project/:projectId -- all bids on a project
router.get('/:project/:projectId', verifyToken, async (req, res) => {
    try {
        const result = await runQuery(
            'SELECT b.bid_id, b.amount, b.delivery_time,
            b.status, b.submitted_at,

```



```

        u.name AS freelancer_name,
        fp.rating, fp.experience_level
    FROM bids b
    INNER JOIN freelancer_profile fp
        ON b.freelancer_id = fp.freelancer_id
    INNER JOIN users u ON fp.freelancer_id = u.user_id
    WHERE b.project_id = :id
    ORDER BY b.amount ASC',
    [req.params.projectId]
);
res.json(result.rows);
} catch (err) { res.status(500).json({ error: err.message }); }
});

// GET /api/bids/freelancer/:freelancerId -- all bids by a freelancer
router.get('/freelancer/:freelancerId', verifyToken, async (req, res)
↳ => {
    try {
        const result = await runQuery(
            'SELECT b.bid_id, b.amount, b.status,
                b.delivery_time, b.submitted_at,
                p.title AS project_title,
                p.budget, p.status AS project_status
            FROM bids b
            INNER JOIN projects p ON b.project_id = p.project_id
            WHERE b.freelancer_id = :id
            ORDER BY b.submitted_at DESC',
            [req.params.freelancerId]
        );
        res.json(result.rows);
    } catch (err) { res.status(500).json({ error: err.message }); }
});

// POST /api/bids -- submit a bid (freelancer only)
router.post('/', verifyToken, requireRole('freelancer'),
    async (req, res) => {
        const { project_id, amount, delivery_time } = req.body;
        try {
            await runQuery(
                'INSERT INTO bids VALUES
                    (seq_bids.nextval, :project_id, :freelancer_id,
                     :amount, :delivery_time, 'pending', SYSDATE)',
                [project_id, req.user.id, amount, delivery_time]
            );
            res.json({ message: 'Bid submitted successfully' });
        }
    }
});

```

```
    } catch (err) { res.status(500).json({ error: err.message }); }
  });

// PUT /api/bids/:id/accept -- accept a bid (transaction)
router.put('/:id/accept', verifyToken, requireRole('client'),
  async (req, res) => {
    let connection;
    try {
      connection = await oracledb.getConnection();

      // mark this bid as accepted
      await connection.execute(
        'UPDATE bids SET status = 'accepted'
          WHERE bid_id = :id',
        [req.params.id], { autoCommit: false }
      );

      // reject all other bids on the same project
      await connection.execute(
        'UPDATE bids SET status = 'rejected'
          WHERE project_id = (
            SELECT project_id FROM bids WHERE bid_id = :id)
          AND bid_id != :id',
        [req.params.id, req.params.id], { autoCommit: false }
      );

      // update project status to in_progress
      await connection.execute(
        'UPDATE projects SET status = 'in_progress'
          WHERE project_id = (
            SELECT project_id FROM bids WHERE bid_id = :id)',
        [req.params.id], { autoCommit: false }
      );

      // create a contract for the accepted bid
      await connection.execute(
        'INSERT INTO contracts VALUES
          (seq_contracts.nextval, :bid_id,
            (SELECT amount FROM bids WHERE bid_id = :bid_id),
            SYSDATE, NULL, 'active')',
        { bid_id: req.params.id }, { autoCommit: false }
      );

      await connection.commit();
      res.json({ message: 'Bid accepted and contract created' });
    } catch (err) { res.status(500).json({ error: err.message }); }
  });
```

```
    } catch (err) {
      if (connection) await connection.rollback();
      res.status(500).json({ error: err.message });
    } finally {
      if (connection) await connection.close();
    }
  });

// DELETE /api/bids/:id -- withdraw a pending bid
router.delete('/:id', verifyToken, requireRole('freelancer'),
  async (req, res) => {
    try {
      await runQuery(
        'DELETE FROM bids
        WHERE bid_id = :id
        AND freelancer_id = :freelancer_id
        AND status = 'pending'',
        [req.params.id, req.user.id]
      );
      res.json({ message: 'Bid withdrawn successfully' });
    } catch (err) { res.status(500).json({ error: err.message }); }
  });

module.exports = router;
```

9.8 Route File: contracts.js

Read and update operations for contracts with JOIN queries.

Listing 9.7: routes/contracts.js

```
const express = require('express');
const router = express.Router();
const { runQuery } = require('../db');
const { verifyToken } = require('../middleware/auth');

// GET /api/contracts/client/:clientId
router.get('/:client/:clientId', verifyToken, async (req, res) => {
  try {
    const result = await runQuery(
      'SELECT c.contract_id, c.amount, c.status,
        c.start_date, c.end_date,
        p.title AS project_title,
        u.name AS freelancer_name
```

```

        FROM contracts c
        INNER JOIN bids b ON c.bid_id = b.bid_id
        INNER JOIN projects p ON b.project_id = p.project_id
        INNER JOIN freelancer_profile fp
            ON b.freelancer_id = fp.freelancer_id
        INNER JOIN users u ON fp.freelancer_id = u.user_id
        WHERE p.client_id = :id
        ORDER BY c.contract_id DESC',
        [req.params.clientId]
    );
    res.json(result.rows);
} catch (err) { res.status(500).json({ error: err.message }); }
});

// GET /api/contracts/freelancer/:freelancerId
router.get('/:freelancer/:freelancerId', verifyToken, async (req, res)
↳ => {
    try {
        const result = await runQuery(
            'SELECT c.contract_id, c.amount, c.status,
                c.start_date, c.end_date,
                p.title AS project_title,
                u.name AS client_name
            FROM contracts c
            INNER JOIN bids b ON c.bid_id = b.bid_id
            INNER JOIN projects p ON b.project_id = p.project_id
            INNER JOIN client_profile cp ON p.client_id = cp.client_id
            INNER JOIN users u ON cp.client_id = u.user_id
            WHERE b.freelancer_id = :id
            ORDER BY c.contract_id DESC',
            [req.params.freelancerId]
        );
        res.json(result.rows);
    } catch (err) { res.status(500).json({ error: err.message }); }
});

// PUT /api/contracts/:id/complete -- mark contract as completed
router.put('/:id/complete', verifyToken, async (req, res) => {
    try {
        await runQuery(
            'UPDATE contracts SET status = 'completed',
                end_date = SYSDATE
            WHERE contract_id = :id',
            [req.params.id]
        );
    }

```

```
    res.json({ message: 'Contract marked as completed' });
  } catch (err) { res.status(500).json({ error: err.message }); }
});

module.exports = router;
```

9.9 Route File: payments.js

Full CRUD for payments. Recording a payment also updates the freelancer rating as part of a transaction.

Listing 9.8: routes/payments.js – full CRUD

```
const express = require('express');
const router = express.Router();
const { runQuery } = require('../db');
const oracledb = require('oracledb');
const { verifyToken, requireRole } = require('../middleware/auth');

// GET /api/payments/contract/:contractId
router.get('/contract/:contractId', verifyToken, async (req, res) => {
  try {
    const result = await runQuery(
      'SELECT payment_id, amount, pay_date,
        method, status
      FROM payments
      WHERE contract_id = :id
      ORDER BY pay_date DESC',
      [req.params.contractId]
    );
    res.json(result.rows);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

// GET /api/payments/freelancer/:freelancerId -- earnings summary
router.get('/freelancer/:freelancerId', verifyToken, async (req, res)
↳ => {
  try {
    const result = await runQuery(
      'SELECT p.title AS project_title,
        pay.amount, pay.pay_date,
        pay.method, pay.status
      FROM payments pay
```

```

        INNER JOIN contracts c ON pay.contract_id = c.contract_id
        INNER JOIN bids b ON c.bid_id = b.bid_id
        INNER JOIN projects p ON b.project_id = p.project_id
        WHERE b.freelancer_id = :id
        ORDER BY pay.pay_date DESC',
        [req.params.freelancerId]
    );
    res.json(result.rows);
} catch (err) { res.status(500).json({ error: err.message }); }
});

// POST /api/payments -- record a payment (client only, transaction)
router.post('/', verifyToken, requireRole('client'), async (req, res)
↳ => {
    const { contract_id, amount, method } = req.body;
    let connection;
    try {
        connection = await oracledb.getConnection();

        await connection.execute(
            'INSERT INTO payments VALUES
            (seq_payments.nextval, :contract_id,
            :amount, SYSDATE, :method, 'paid')',
            { contract_id, amount, method },
            { autoCommit: false }
        );

        // send notification to freelancer
        await connection.execute(
            'INSERT INTO notifications VALUES
            (seq_notifications.nextval,
            (SELECT b.freelancer_id FROM contracts c
            INNER JOIN bids b ON c.bid_id = b.bid_id
            WHERE c.contract_id = :contract_id),
            'Payment of ' || :amount || ' received',
            SYSDATE, 'N', 'payment')',
            { contract_id, amount },
            { autoCommit: false }
        );

        await connection.commit();
        res.json({ message: 'Payment recorded successfully' });
    } catch (err) {
        if (connection) await connection.rollback();
        res.status(500).json({ error: err.message });
    }
}

```

```
    } finally {
      if (connection) await connection.close();
    }
  });

// DELETE /api/payments/:id -- delete a pending payment
router.delete('/:id', verifyToken, requireRole('client'),
  async (req, res) => {
    try {
      await runQuery(
        `DELETE FROM payments
         WHERE payment_id = :id AND status = 'pending'`,
        [req.params.id]
      );
      res.json({ message: 'Payment deleted' });
    } catch (err) { res.status(500).json({ error: err.message }); }
  });

module.exports = router;
```

9.10 Route File: reviews.js

Listing 9.9: routes/reviews.js – full CRUD

```
const express = require('express');
const router = express.Router();
const { runQuery } = require('../db');
const { verifyToken } = require('../middleware/auth');

// GET /api/reviews/freelancer/:freelancerId
router.get('/:freelancer/:freelancerId', verifyToken, async (req, res)
  => {
    try {
      const result = await runQuery(
        `SELECT r.review_id, r.rating, r.comment,
              r.created_at,
              u.name AS reviewer_name,
              p.title AS project_title
         FROM reviews r
        INNER JOIN users u ON r.reviewer_id = u.user_id
        INNER JOIN contracts c ON r.contract_id = c.contract_id
        INNER JOIN bids b ON c.bid_id = b.bid_id
        INNER JOIN projects p ON b.project_id = p.project_id`
      );
    }
  });
```

```
        WHERE r.reviewee_id = :id
        ORDER BY r.created_at DESC',
        [req.params.freelancerId]
    );
    res.json(result.rows);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

// POST /api/reviews -- submit a review
router.post('/', verifyToken, async (req, res) => {
  const { contract_id, reviewee_id, rating, comment } = req.body;
  try {
    await runQuery(
      'INSERT INTO reviews VALUES
      (seq_reviews.nextval, :contract_id,
       :reviewer_id, :reviewee_id,
       :rating, :comment, SYSDATE)',
      [contract_id, req.user.id, reviewee_id, rating, comment]
    );
    // update freelancer average rating
    await runQuery(
      'UPDATE freelancer_profile SET rating = (
        SELECT ROUND(AVG(rating), 2) FROM reviews
        WHERE reviewee_id = :id)
      WHERE freelancer_id = :id',
      [reviewee_id, reviewee_id]
    );
    res.json({ message: 'Review submitted successfully' });
  } catch (err) { res.status(500).json({ error: err.message }); }
});

// DELETE /api/reviews/:id
router.delete('/:id', verifyToken, async (req, res) => {
  try {
    await runQuery(
      'DELETE FROM reviews
      WHERE review_id = :id AND reviewer_id = :reviewer_id',
      [req.params.id, req.user.id]
    );
    res.json({ message: 'Review deleted' });
  } catch (err) { res.status(500).json({ error: err.message }); }
});

module.exports = router;
```


9.11 Route File: disputes.js

Listing 9.10: routes/disputes.js – full CRUD

```
const express = require('express');
const router = express.Router();
const { runQuery } = require('../db');
const { verifyToken } = require('../middleware/auth');

// GET /api/disputes/contract/:contractId
router.get('/contract/:contractId', verifyToken, async (req, res) => {
  try {
    const result = await runQuery(
      'SELECT d.dispute_id, d.reason, d.status,
        d.created_at, u.name AS raised_by_name
      FROM disputes d
      INNER JOIN users u ON d.raised_by = u.user_id
      WHERE d.contract_id = :id',
      [req.params.contractId]
    );
    res.json(result.rows);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

// POST /api/disputes -- raise a dispute
router.post('/', verifyToken, async (req, res) => {
  const { contract_id, reason } = req.body;
  try {
    await runQuery(
      'INSERT INTO disputes VALUES
        (seq_disputes.nextval, :contract_id,
        :raised_by, :reason, 'open', SYSDATE)',
      [contract_id, req.user.id, reason]
    );
    await runQuery(
      'UPDATE contracts SET status = 'disputed'
      WHERE contract_id = :id',
      [contract_id]
    );
    res.json({ message: 'Dispute raised successfully' });
  } catch (err) { res.status(500).json({ error: err.message }); }
});

// PUT /api/disputes/:id -- update dispute status (admin)
```

```
router.put('/:id', verifyToken, async (req, res) => {
  const { status } = req.body;
  try {
    await runQuery(
      'UPDATE disputes SET status = :status
      WHERE dispute_id = :id',
      [status, req.params.id]
    );
    res.json({ message: 'Dispute status updated' });
  } catch (err) { res.status(500).json({ error: err.message }); }
});

module.exports = router;
```

9.12 Route File: notifications.js

Listing 9.11: routes/notifications.js – full CRUD

```
const express = require('express');
const router = express.Router();
const { runQuery } = require('../db');
const { verifyToken } = require('../middleware/auth');

// GET /api/notifications/:userId -- all notifications for a user
router.get('/:userId', verifyToken, async (req, res) => {
  try {
    const result = await runQuery(
      'SELECT notification_id, message,
      date_sent, is_read, type
      FROM notifications
      WHERE user_id = :id
      ORDER BY date_sent DESC',
      [req.params.userId]
    );
    res.json(result.rows);
  } catch (err) { res.status(500).json({ error: err.message }); }
});

// PUT /api/notifications/:id/read -- mark as read
router.put('/:id/read', verifyToken, async (req, res) => {
  try {
    await runQuery(
      'UPDATE notifications SET is_read = 'Y'
```

```

        WHERE notification_id = :id',
        [req.params.id]
    );
    res.json({ message: 'Notification marked as read' });
  } catch (err) { res.status(500).json({ error: err.message }); }
});

// DELETE /api/notifications/:id
router.delete('/:id', verifyToken, async (req, res) => {
  try {
    await runQuery(
      'DELETE FROM notifications
      WHERE notification_id = :id AND user_id = :user_id',
      [req.params.id, req.user.id]
    );
    res.json({ message: 'Notification deleted' });
  } catch (err) { res.status(500).json({ error: err.message }); }
});

module.exports = router;

```

9.13 Route File: admin.js

Admin has read-only access to all 15 tables plus aggregate summary queries.

Listing 9.12: routes/admin.js – all tables and summaries

```

const express = require('express');
const router = express.Router();
const { runQuery } = require('../db');
const { verifyToken, requireRole } = require('../middleware/auth');

const admin = [verifyToken, requireRole('admin')];

// GET all users
router.get('/users', ...admin, async (req, res) => {
  const r = await runQuery(
    'SELECT u.user_id, u.name, u.email, u.phone,
        u.created_at,
        CASE WHEN cp.client_id IS NOT NULL
              THEN 'client' ELSE 'freelancer' END AS role
    FROM users u
    LEFT JOIN client_profile cp ON u.user_id = cp.client_id
    ORDER BY u.user_id'
  );
  res.json(r);
});

```

```
);
res.json(r.rows);
});

// GET all projects with client name
router.get('/projects', ...admin, async (req, res) => {
  const r = await runQuery(
    'SELECT p.*, u.name AS client_name
    FROM projects p
    INNER JOIN client_profile cp ON p.client_id = cp.client_id
    INNER JOIN users u ON cp.client_id = u.user_id
    ORDER BY p.project_id DESC'
  );
  res.json(r.rows);
});

// GET all bids with names
router.get('/bids', ...admin, async (req, res) => {
  const r = await runQuery(
    'SELECT b.bid_id, b.amount, b.status,
           p.title AS project_title,
           u.name AS freelancer_name
    FROM bids b
    INNER JOIN projects p ON b.project_id = p.project_id
    INNER JOIN freelancer_profile fp
      ON b.freelancer_id = fp.freelancer_id
    INNER JOIN users u ON fp.freelancer_id = u.user_id
    ORDER BY b.bid_id DESC'
  );
  res.json(r.rows);
});

// GET all contracts with both parties
router.get('/contracts', ...admin, async (req, res) => {
  const r = await runQuery(
    'SELECT c.contract_id, c.amount, c.status,
           p.title AS project_title,
           uc.name AS client_name,
           uf.name AS freelancer_name
    FROM contracts c
    INNER JOIN bids b ON c.bid_id = b.bid_id
    INNER JOIN projects p ON b.project_id = p.project_id
    INNER JOIN client_profile cp ON p.client_id = cp.client_id
    INNER JOIN users uc ON cp.client_id = uc.user_id
    INNER JOIN freelancer_profile fp
```

```
        ON b.freelancer_id = fp.freelancer_id
        INNER JOIN users uf ON fp.freelancer_id = uf.user_id
        ORDER BY c.contract_id DESC '
    );
    res.json(r.rows);
});

// GET all payments
router.get('/payments', ...admin, async (req, res) => {
    const r = await runQuery(
        'SELECT pay.*, p.title AS project_title
        FROM payments pay
        INNER JOIN contracts c ON pay.contract_id = c.contract_id
        INNER JOIN bids b ON c.bid_id = b.bid_id
        INNER JOIN projects p ON b.project_id = p.project_id
        ORDER BY pay.pay_date DESC '
    );
    res.json(r.rows);
});

// GET all reviews
router.get('/reviews', ...admin, async (req, res) => {
    const r = await runQuery(
        'SELECT r.review_id, r.rating, r.comment,
            u1.name AS reviewer, u2.name AS reviewee
        FROM reviews r
        INNER JOIN users u1 ON r.reviewer_id = u1.user_id
        INNER JOIN users u2 ON r.reviewee_id = u2.user_id
        ORDER BY r.created_at DESC '
    );
    res.json(r.rows);
});

// GET all disputes
router.get('/disputes', ...admin, async (req, res) => {
    const r = await runQuery(
        'SELECT d.*, u.name AS raised_by_name
        FROM disputes d
        INNER JOIN users u ON d.raised_by = u.user_id
        ORDER BY d.created_at DESC '
    );
    res.json(r.rows);
});

// GET system summary
```

```
router.get('/summary', ...admin, async (req, res) => {
  const r = await runQuery(
    'SELECT
      (SELECT COUNT(*) FROM users) AS total_users,
      (SELECT COUNT(*) FROM projects) AS total_projects,
      (SELECT COUNT(*) FROM bids) AS total_bids,
      (SELECT COUNT(*) FROM contracts) AS total_contracts,
      (SELECT SUM(amount) FROM payments
        WHERE status = 'paid') AS total_revenue,
      (SELECT COUNT(*) FROM disputes
        WHERE status = 'open') AS open_disputes
    FROM dual '
  );
  res.json(r.rows[0]);
});

module.exports = router;
```

9.14 API Endpoints Summary

Table 9.1: All 35 API Endpoints

Method	Endpoint	Role	Action
POST	/api/auth/register	Public	Register user
POST	/api/auth/login	Public	Login and get token
GET	/api/projects	All	All projects with client
GET	/api/projects/:id	All	Single project details
GET	/api/projects/client/:id	Client	Client's projects
POST	/api/projects	Client	Create project
PUT	/api/projects/:id	Client	Update project
DELETE	/api/projects/:id	Client	Delete project
GET	/api/bids/project/:id	Client	Bids on a project
GET	/api/bids/freelancer/:id	Freelancer	My bids
POST	/api/bids	Freelancer	Submit bid
PUT	/api/bids/:id/accept	Client	Accept bid (transaction)
DELETE	/api/bids/:id	Freelancer	Withdraw bid
GET	/api/contracts/client/:id	Client	My contracts
GET	/api/contracts/freelancer/:id	Freelancer	My contracts
PUT	/api/contracts/:id/complete	All	Complete contract
GET	/api/payments/contract/:id	All	Payments on contract
GET	/api/payments/freelancer/:id	Freelancer	Earnings
POST	/api/payments	Client	Record payment (transaction)
DELETE	/api/payments/:id	Client	Delete pending payment
GET	/api/reviews/freelancer/:id	All	Reviews for freelancer
POST	/api/reviews	All	Submit review
DELETE	/api/reviews/:id	All	Delete own review
GET	/api/disputes/contract/:id	All	Disputes on contract
POST	/api/disputes	All	Raise dispute
PUT	/api/disputes/:id	Admin	Update dispute status
GET	/api/notifications/:id	All	My notifications
PUT	/api/notifications/:id/read	All	Mark as read
DELETE	/api/notifications/:id	All	Delete notification
GET	/api/admin/users	Admin	All users
GET	/api/admin/projects	Admin	All projects
GET	/api/admin/bids	Admin	All bids
GET	/api/admin/contracts	Admin	All contracts
GET	/api/admin/payments	Admin	All payments
GET	/api/admin/summary	Admin	System summary

Chapter 10

Frontend Architecture

10.1 Overview

The SkillLink frontend is a React.js single-page application. It communicates with the backend exclusively through Axios HTTP requests. The interface is divided into three role-based dashboards: Client, Freelancer, and Admin. React Router DOM protects all routes so that unauthenticated users are redirected to the login page and authenticated users can only access pages matching their role.

10.2 Project Structure

Listing 10.1: Frontend folder structure

```
skilllink-frontend/  
  src/  
    components/  
      Navbar.jsx  
      ProtectedRoute.jsx  
    context/  
      AuthContext.jsx  
    pages/  
      Login.jsx  
      Register.jsx  
    client/  
      ClientDashboard.jsx  
      PostProject.jsx  
      ViewBids.jsx  
      MyContracts.jsx  
      MakePayment.jsx
```



```
freelancer/  
  FreelancerDashboard.jsx  
  BrowseProjects.jsx  
  SubmitBid.jsx  
  MyBids.jsx  
  MyContracts.jsx  
  Earnings.jsx  
admin/  
  AdminDashboard.jsx  
  AllUsers.jsx  
  AllProjects.jsx  
  AllContracts.jsx  
  AllPayments.jsx  
  SystemSummary.jsx  
api/  
  axios.js  
App.jsx  
main.jsx
```

10.3 Axios Configuration

A shared Axios instance attaches the JWT token to every request automatically.

Listing 10.2: api/axios.js

```
import axios from 'axios';  
  
const API = axios.create({  
  baseURL: 'http://localhost:5000/api'  
});  
  
// attach token to every request  
API.interceptors.request.use((config) => {  
  const token = localStorage.getItem('token');  
  if (token) config.headers.Authorization = 'Bearer ${token}';  
  return config;  
});  
  
export default API;
```

10.4 Authentication Context

The AuthContext stores the logged-in user and token globally so every component can access them without prop drilling.

Listing 10.3: context/AuthContext.jsx

```
import { createContext, useState, useContext } from 'react';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(
    JSON.parse(localStorage.getItem('user')) || null
  );

  const login = (userData, token) => {
    localStorage.setItem('token', token);
    localStorage.setItem('user', JSON.stringify(userData));
    setUser(userData);
  };

  const logout = () => {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

10.5 Protected Route Component

Redirects unauthenticated users to login. Redirects authenticated users to their correct dashboard if they try to access a wrong role's page.

Listing 10.4: components/ProtectedRoute.jsx

```
import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
```

```

export default function ProtectedRoute({ children, role }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" />;
  if (role && user.role !== role)
    return <Navigate to={`/${user.role}`} />;
  return children;
}

```

10.6 App.jsx — Route Definitions

All routes are defined here with role-based protection applied to every dashboard page.

Listing 10.5: App.jsx – all routes

```

import { BrowserRouter, Routes, Route, Navigate }
  from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import ProtectedRoute from '../components/ProtectedRoute';

import Login          from '../pages/Login';
import Register        from '../pages/Register';

import ClientDashboard  from '../pages/client/ClientDashboard';
import PostProject      from '../pages/client/PostProject';
import ViewBids         from '../pages/client/ViewBids';
import MyContractsClient from '../pages/client/MyContracts';
import MakePayment      from '../pages/client/MakePayment';

import FreelancerDashboard from
  '../pages/freelancer/FreelancerDashboard';
import BrowseProjects  from '../pages/freelancer/BrowseProjects';
import SubmitBid       from '../pages/freelancer/SubmitBid';
import MyBids          from '../pages/freelancer/MyBids';
import MyContractsFL   from '../pages/freelancer/MyContracts';
import Earnings        from '../pages/freelancer/Earnings';

import AdminDashboard  from '../pages/admin/AdminDashboard';
import AllUsers        from '../pages/admin/AllUsers';
import AllProjects     from '../pages/admin/AllProjects';
import AllContracts    from '../pages/admin/AllContracts';
import AllPayments     from '../pages/admin/AllPayments';
import SystemSummary   from '../pages/admin/SystemSummary';

```

```
export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<Navigate to="/login" />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />

          {/* Client routes */}
          <Route path="/client" element={
            <ProtectedRoute role="client">
              <ClientDashboard />
            </ProtectedRoute>} />
          <Route path="/client/post-project" element={
            <ProtectedRoute role="client">
              <PostProject />
            </ProtectedRoute>} />
          <Route path="/client/bids/:projectId" element={
            <ProtectedRoute role="client">
              <ViewBids />
            </ProtectedRoute>} />
          <Route path="/client/contracts" element={
            <ProtectedRoute role="client">
              <MyContractsClient />
            </ProtectedRoute>} />
          <Route path="/client/payment/:contractId" element={
            <ProtectedRoute role="client">
              <MakePayment />
            </ProtectedRoute>} />

          {/* Freelancer routes */}
          <Route path="/freelancer" element={
            <ProtectedRoute role="freelancer">
              <FreelancerDashboard />
            </ProtectedRoute>} />
          <Route path="/freelancer/browse" element={
            <ProtectedRoute role="freelancer">
              <BrowseProjects />
            </ProtectedRoute>} />
          <Route path="/freelancer/bid/:projectId" element={
            <ProtectedRoute role="freelancer">
              <SubmitBid />
            </ProtectedRoute>} />
          <Route path="/freelancer/my-bids" element={
```

```
        <ProtectedRoute role="freelancer">
            <MyBids />
        </ProtectedRoute>} />
    <Route path="/freelancer/contracts" element={
        <ProtectedRoute role="freelancer">
            <MyContractsFL />
        </ProtectedRoute>} />
    <Route path="/freelancer/earnings" element={
        <ProtectedRoute role="freelancer">
            <Earnings />
        </ProtectedRoute>} />

    {/* Admin routes */}
    <Route path="/admin" element={
        <ProtectedRoute role="admin">
            <AdminDashboard />
        </ProtectedRoute>} />
    <Route path="/admin/users" element={
        <ProtectedRoute role="admin">
            <AllUsers />
        </ProtectedRoute>} />
    <Route path="/admin/projects" element={
        <ProtectedRoute role="admin">
            <AllProjects />
        </ProtectedRoute>} />
    <Route path="/admin/contracts" element={
        <ProtectedRoute role="admin">
            <AllContracts />
        </ProtectedRoute>} />
    <Route path="/admin/payments" element={
        <ProtectedRoute role="admin">
            <AllPayments />
        </ProtectedRoute>} />
    <Route path="/admin/summary" element={
        <ProtectedRoute role="admin">
            <SystemSummary />
        </ProtectedRoute>} />
    </Routes>
    </BrowserRouter>
    </AuthProvider>
);
}
```

10.7 Login Page

Listing 10.6: pages/Login.jsx

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import API from '../api/axios';

export default function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleLogin = async (e) => {
    e.preventDefault();
    try {
      const res = await API.post('/auth/login', { email, password });
      login(res.data.user, res.data.token);
      navigate(`/${res.data.user.role}`);
    } catch (err) {
      setError('Invalid email or password');
    }
  };

  return (
    <div className="login-container">
      <h2>SkillLink Login</h2>
      {error && <p className="error">{error}</p>}
      <form onSubmit={handleLogin}>
        <input type="email" placeholder="Email"
          value={email}
          onChange={e => setEmail(e.target.value)} required />
        <input type="password" placeholder="Password"
          value={password}
          onChange={e => setPassword(e.target.value)} required />
        <button type="submit">Login</button>
      </form>
    </div>
  );
}
```

10.8 Client Pages

10.8.1 PostProject.jsx

Listing 10.7: pages/client/PostProject.jsx

```
import { useState } from 'react';
import API from '../../api/axios';
import { useAuth } from '../../context/AuthContext';

export default function PostProject() {
  const { user } = useAuth();
  const [form, setForm] = useState({
    title: '', description: '',
    budget: '', deadline: ''
  });

  const handleSubmit = async (e) => {
    e.preventDefault();
    await API.post('/projects', form);
    alert('Project posted successfully');
  };

  return (
    <div>
      <h2>Post a New Project</h2>
      <form onSubmit={handleSubmit}>
        <input placeholder="Title"
          onChange={e =>
            setForm({...form, title: e.target.value})} />
        <textarea placeholder="Description"
          onChange={e =>
            setForm({...form, description: e.target.value})} />
        <input type="number" placeholder="Budget"
          onChange={e =>
            setForm({...form, budget: e.target.value})} />
        <input type="date"
          onChange={e =>
            setForm({...form, deadline: e.target.value})} />
        <button type="submit">Post Project</button>
      </form>
    </div>
  );
}
```

10.8.2 ViewBids.jsx

Listing 10.8: pages/client/ViewBids.jsx

```
import { useEffect, useState } from 'react';
import { useParams } from 'react-router-dom';
import API from '../../api/axios';

export default function ViewBids() {
  const { projectId } = useParams();
  const [bids, setBids] = useState([]);

  useEffect(() => {
    API.get(`/bids/project/${projectId}`)
      .then(res => setBids(res.data));
  }, [projectId]);

  const acceptBid = async (bidId) => {
    await API.put(`/bids/${bidId}/accept`);
    alert('Bid accepted and contract created');
  };

  return (
    <div>
      <h2>Bids on Project</h2>
      {bids.map(bid => (
        <div key={bid.BID_ID} className="bid-card">
          <p>{bid.FREELANCER_NAME} Rs. {bid.AMOUNT}</p>
          <p>Rating: {bid.RATING} | {bid.EXPERIENCE_LEVEL}</p>
          <p>Delivery: {bid.DELIVERY_TIME} days</p>
          <button onClick={() => acceptBid(bid.BID_ID)}>
            Accept Bid
          </button>
        </div>
      ))}
    </div>
  );
}
```

10.8.3 MakePayment.jsx

Listing 10.9: pages/client/MakePayment.jsx

```
import { useState } from 'react';
import { useParams } from 'react-router-dom';
import API from '../../api/axios';
```



```
export default function MakePayment() {
  const { contractId } = useParams();
  const [amount, setAmount] = useState('');
  const [method, setMethod] = useState('bank');

  const handlePayment = async (e) => {
    e.preventDefault();
    await API.post('/payments', {
      contract_id: contractId, amount, method
    });
    alert('Payment recorded successfully');
  };

  return (
    <div>
      <h2>Release Payment</h2>
      <form onSubmit={handlePayment}>
        <input type="number" placeholder="Amount"
          onChange={e => setAmount(e.target.value)} />
        <select onChange={e => setMethod(e.target.value)}>
          <option value="bank">Bank Transfer</option>
          <option value="easypaisa">Easypaisa</option>
          <option value="jazzcash">JazzCash</option>
          <option value="card">Card</option>
        </select>
        <button type="submit">Release Payment</button>
      </form>
    </div>
  );
}
```

10.9 Freelancer Pages

10.9.1 BrowseProjects.jsx

Listing 10.10: pages/freelancer/BrowseProjects.jsx

```
import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import API from '../../api/axios';

export default function BrowseProjects() {
  const [projects, setProjects] = useState([]);
  const navigate = useNavigate();
```

```
useEffect(() => {
  API.get('/projects').then(res =>
    setProjects(res.data.filter(p =>
      p.STATUS === 'open'))
  );
}, []);

return (
  <div>
    <h2>Open Projects</h2>
    {projects.map(p => (
      <div key={p.PROJECT_ID} className="project-card">
        <h3>{p.TITLE}</h3>
        <p>Budget: Rs. {p.BUDGET}</p>
        <p>Client: {p.CLIENT_NAME}</p>
        <button onClick={() =>
          navigate(`/freelancer/bid/${p.PROJECT_ID}`)}>
          Submit Bid
        </button>
      </div>
    ))}
  </div>
);
}
```

10.9.2 SubmitBid.jsx

Listing 10.11: pages/freelancer/SubmitBid.jsx

```
import { useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import API from '../api/axios';

export default function SubmitBid() {
  const { projectId } = useParams();
  const navigate = useNavigate();
  const [amount, setAmount] = useState('');
  const [days, setDays] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    await API.post('/bids', {
      project_id: projectId,
      amount,
      delivery_time: days
    });
  };
}
```

```
    alert('Bid submitted successfully');
    navigate('/freelancer/my-bids');
  };

  return (
    <div>
      <h2>Submit a Bid</h2>
      <form onSubmit={handleSubmit}>
        <input type="number" placeholder="Your bid amount"
          onChange={e => setAmount(e.target.value)} />
        <input type="number" placeholder="Delivery in days"
          onChange={e => setDays(e.target.value)} />
        <button type="submit">Submit Bid</button>
      </form>
    </div>
  );
}
```

10.9.3 Earnings.jsx

Listing 10.12: pages/freelancer/Earnings.jsx

```
import { useEffect, useState } from 'react';
import API from '../../api/axios';
import { useAuth } from '../../context/AuthContext';

export default function Earnings() {
  const { user } = useAuth();
  const [payments, setPayments] = useState([]);
  const [total, setTotal] = useState(0);

  useEffect(() => {
    API.get(`/payments/freelancer/${user.id}`)
      .then(res => {
        setPayments(res.data);
        const sum = res.data.reduce(
          (acc, p) => acc + p.AMOUNT, 0);
        setTotal(sum);
      });
  }, []);

  return (
    <div>
      <h2>My Earnings</h2>
      <h3>Total Earned: Rs. {total}</h3>
      {payments.map((p, i) => (
```

```
        <div key={i} className="payment-card">
          <p>{p.PROJECT_TITLE}      Rs. {p.AMOUNT}</p>
          <p>Method: {p.METHOD} | Status: {p.STATUS}</p>
        </div>
      )})
    </div>
  );
}
```

10.10 Admin Pages

10.10.1 SystemSummary.jsx

Listing 10.13: pages/admin/SystemSummary.jsx

```
import { useEffect, useState } from 'react';
import API from '../../api/axios';

export default function SystemSummary() {
  const [summary, setSummary] = useState({});

  useEffect(() => {
    API.get('/admin/summary').then(res =>
      setSummary(res.data));
  }, []);

  return (
    <div>
      <h2>System Summary</h2>
      <div className="summary-grid">
        <div><h3>Total Users</h3>
        <p>{summary.TOTAL_USERS}</p></div>
        <div><h3>Total Projects</h3>
        <p>{summary.TOTAL_PROJECTS}</p></div>
        <div><h3>Total Contracts</h3>
        <p>{summary.TOTAL_CONTRACTS}</p></div>
        <div><h3>Total Revenue</h3>
        <p>Rs. {summary.TOTAL_REVENUE}</p></div>
        <div><h3>Open Disputes</h3>
        <p>{summary.OPEN_DISPUTES}</p></div>
      </div>
    </div>
  );
}
```

10.10.2 AllUsers.jsx

Listing 10.14: pages/admin/AllUsers.jsx

```
import { useEffect, useState } from 'react';
import API from '../../api/axios';

export default function AllUsers() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    API.get('/admin/users').then(res => setUsers(res.data));
  }, []);

  return (
    <div>
      <h2>All Users</h2>
      <table>
        <thead>
          <tr>
            <th>ID</th><th>Name</th>
            <th>Email</th><th>Role</th>
          </tr>
        </thead>
        <tbody>
          {users.map(u => (
            <tr key={u.USER_ID}>
              <td>{u.USER_ID}</td>
              <td>{u.NAME}</td>
              <td>{u.EMAIL}</td>
              <td>{u.ROLE}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

10.11 Role-Based Access Summary

Table 10.1: Frontend Pages by Role

Role	Page	Function
Client	ClientDashboard	View own projects and stats
Client	PostProject	Create a new project
Client	ViewBids	See bids and accept one
Client	MyContracts	View active contracts
Client	MakePayment	Release payment to freelancer
Freelancer	FreelancerDashboard	View stats and notifications
Freelancer	BrowseProjects	Browse all open projects
Freelancer	SubmitBid	Place a bid on a project
Freelancer	MyBids	Track submitted bids
Freelancer	MyContracts	View active contracts
Freelancer	Earnings	View payment history and total
Admin	AdminDashboard	Overview of system
Admin	AllUsers	View all registered users
Admin	AllProjects	View all projects
Admin	AllContracts	View all contracts
Admin	AllPayments	View all payments
Admin	SystemSummary	Aggregate stats across system

Summary: The frontend has 17 pages across 3 roles. Every page is protected by the ProtectedRoute component. All data is fetched from the backend via Axios with the JWT token attached automatically. No page writes directly to the database.

Chapter 11

User Interface

11.1 Overview

This chapter presents the complete visual interface of the SkillLink system. The frontend is built in React.js and is divided into three role-based sections: Client, Freelancer, and Admin. Every page shown here is protected by the ProtectedRoute component and is only accessible to users with the correct role. Screenshots were captured from the live running application connected to the Oracle XE backend.

11.2 Authentication Pages

11.2.1 Login Page

The login page is the entry point for all users. It accepts an email and password, calls the `/api/auth/login` endpoint, and redirects the user to their role-specific dashboard automatically.

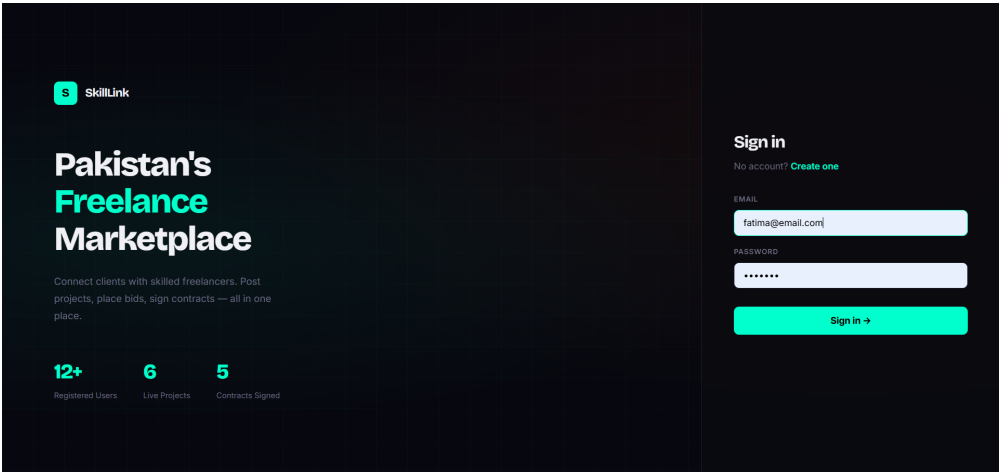


Figure 11.1: SkillLink Login Page

11.2.2 Register Page

The registration page collects the user’s name, email, password, phone, and role. If the user selects Client, company name and location fields appear. If they select Freelancer, bio, hourly rate, and experience level fields appear instead. On submission, a multi-table transaction inserts into `users` and the appropriate subtype table atomically.

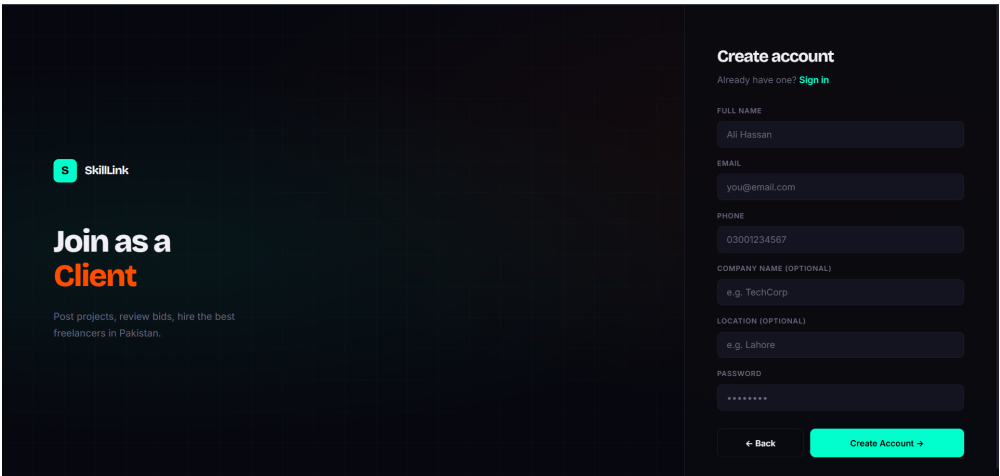


Figure 11.2: SkillLink Register Page

11.3 Client Pages

11.3.1 Client Dashboard

The Client Dashboard is the first page a client sees after logging in. It displays the client’s active projects, the number of bids received on each, and quick links to post a new project or view bids.

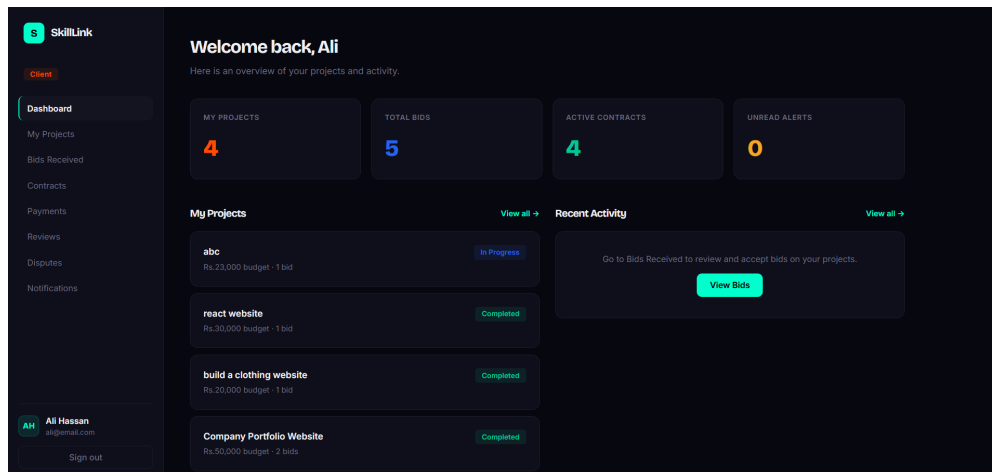


Figure 11.3: Client Dashboard

11.3.2 Post Project

This page allows the client to create a new project by filling in the title, description, budget, and deadline. On submission, the form calls `POST /api/projects` and the new project is inserted into the `projects` table with a status of `open`.

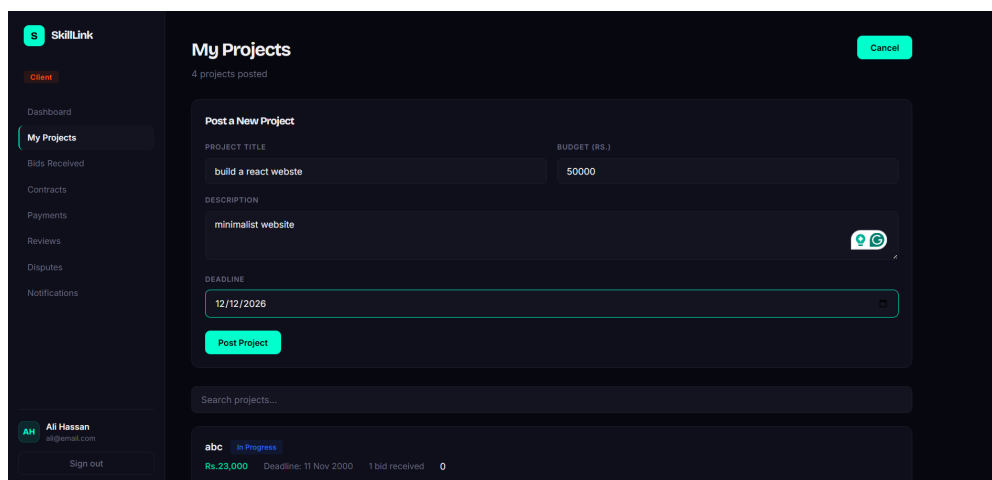


Figure 11.4: Post a New Project

11.3.3 View Bids

When the client clicks on a project, this page shows all bids submitted by freelancers. Each card displays the freelancer's name, bid amount, delivery time, rating, and experience level. The client can accept a bid from this page, which triggers the bid acceptance transaction.

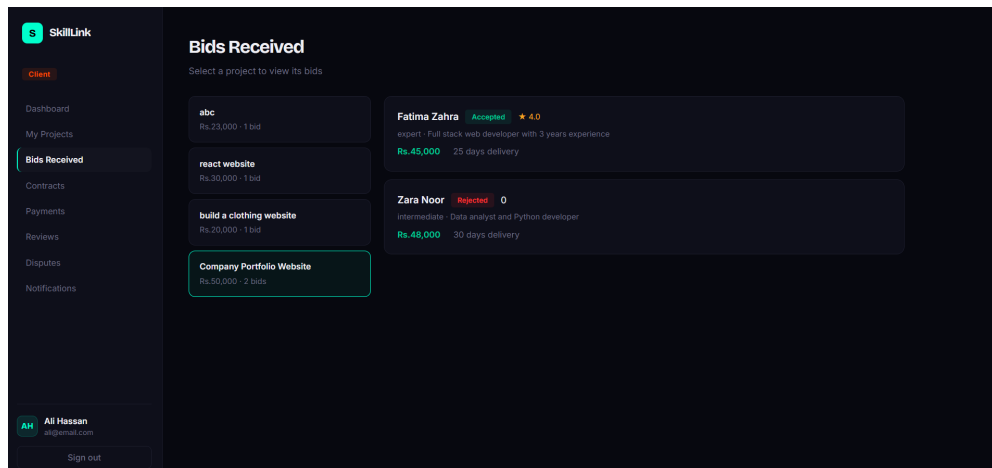


Figure 11.5: View Bids on a Project

11.3.4 My Contracts (Client)

Displays all active and completed contracts for the logged-in client, showing the project title, freelancer name, contract amount, and current status.

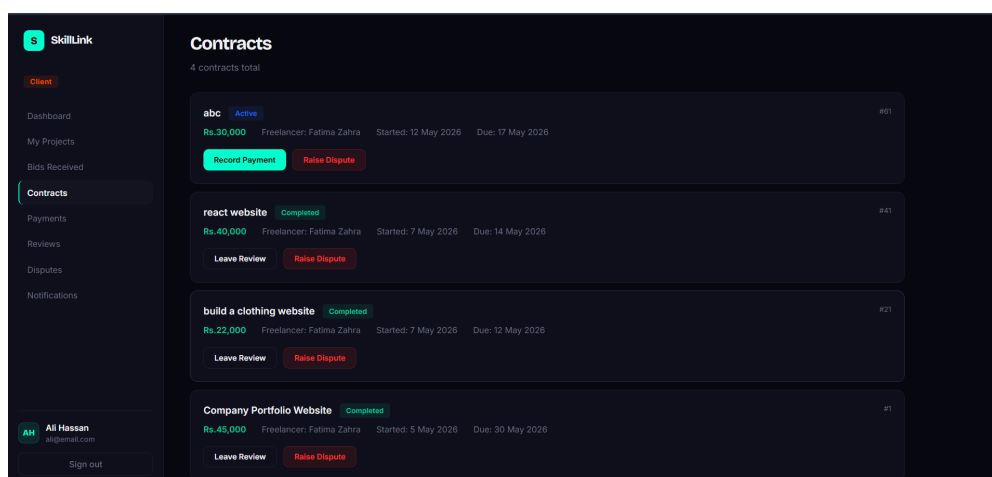


Figure 11.6: Client — My Contracts

11.3.5 Make Payment

The payment page allows the client to release a payment against an active contract. The client enters the amount and selects the payment method (Bank Transfer, Easypaisa, JazzCash, or Card). On submission, the payment is recorded and a notification is automatically sent to the freelancer.

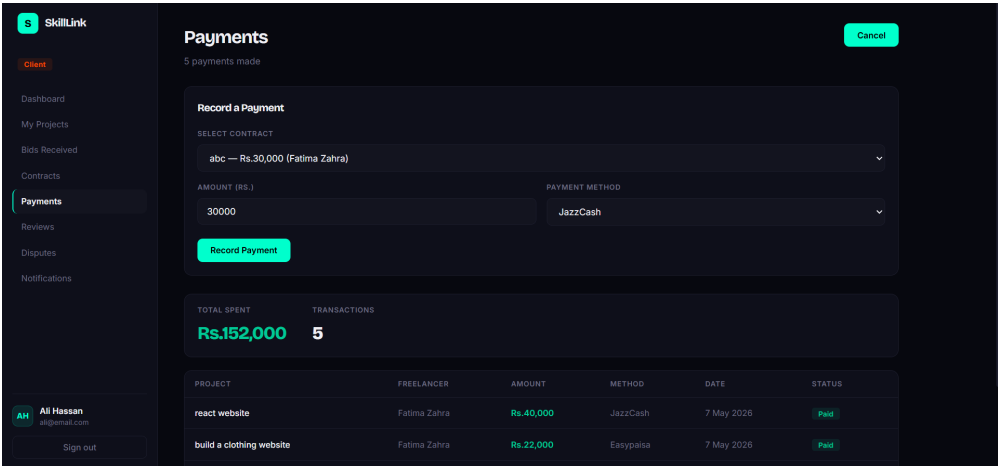


Figure 11.7: Release Payment to Freelancer

11.4 Freelancer Pages

11.4.1 Freelancer Dashboard

The Freelancer Dashboard shows the freelancer’s profile stats, active contracts, and any unread notifications. It provides navigation links to browse projects, view submitted bids, and check earnings.

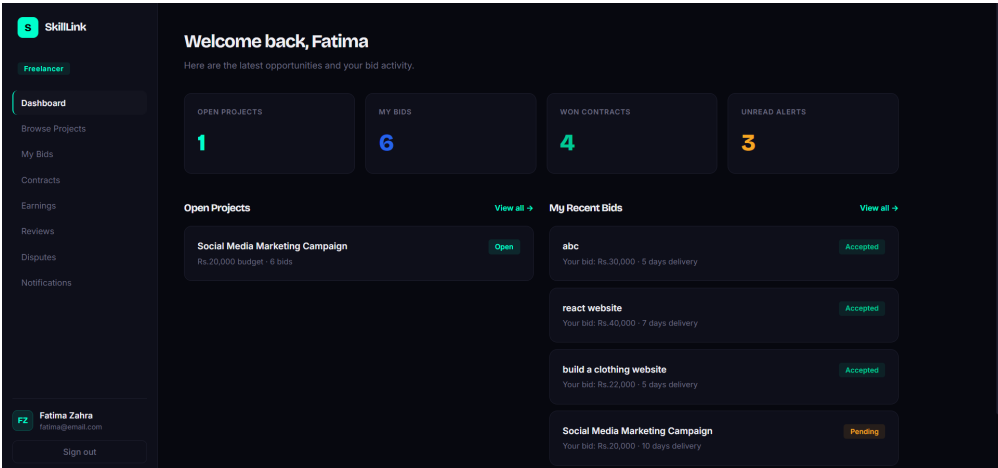


Figure 11.8: Freelancer Dashboard

11.4.2 Browse Projects

Displays all projects with a status of **open**. Each card shows the project title, budget, and the client’s name. The freelancer can click Submit Bid on any project to navigate to the bid submission form.

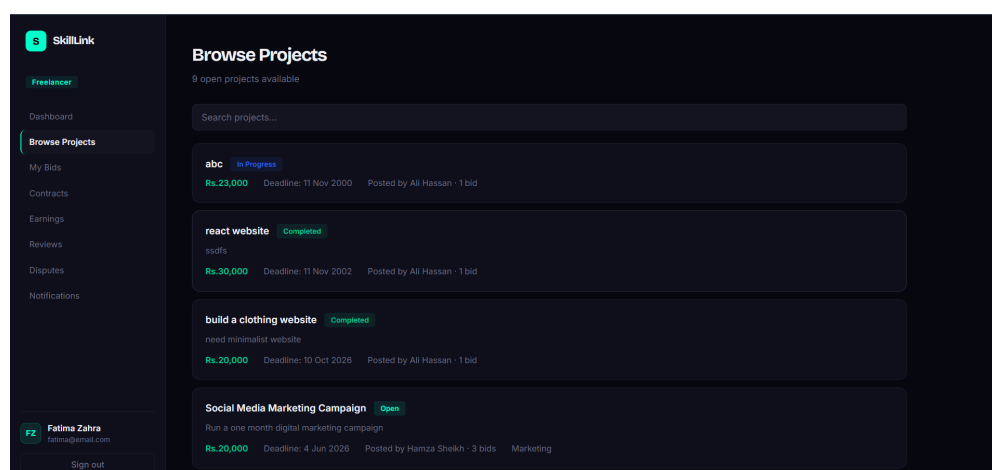


Figure 11.9: Browse Open Projects

11.4.3 Submit Bid

The bid submission form takes the freelancer's proposed amount and delivery time in days. On submission it calls `POST /api/bids` and the bid is inserted into the `bids` table with a status of `pending`.

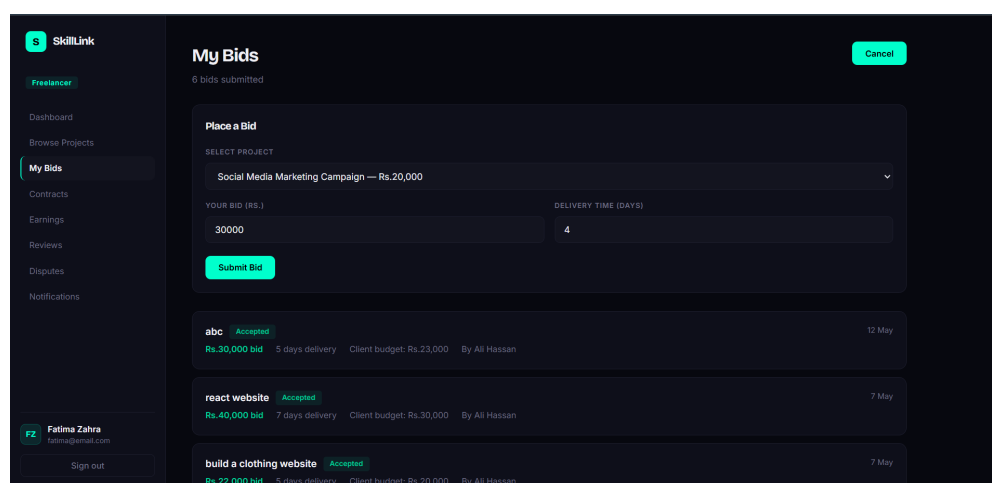


Figure 11.10: Submit a Bid on a Project

11.4.4 My Bids

Shows all bids the freelancer has submitted, with the project title, bid amount, delivery time, and current status (pending, accepted, or rejected).

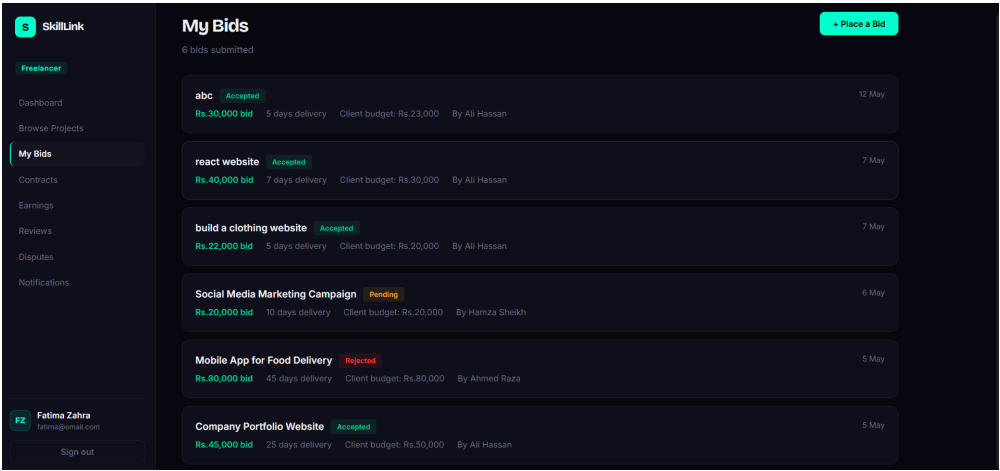


Figure 11.11: Freelancer — My Submitted Bids

11.4.5 Earnings

The Earnings page fetches all paid payments linked to the freelancer’s contracts and displays the project title, amount, payment method, and status. A running total at the top shows the freelancer’s overall earnings.

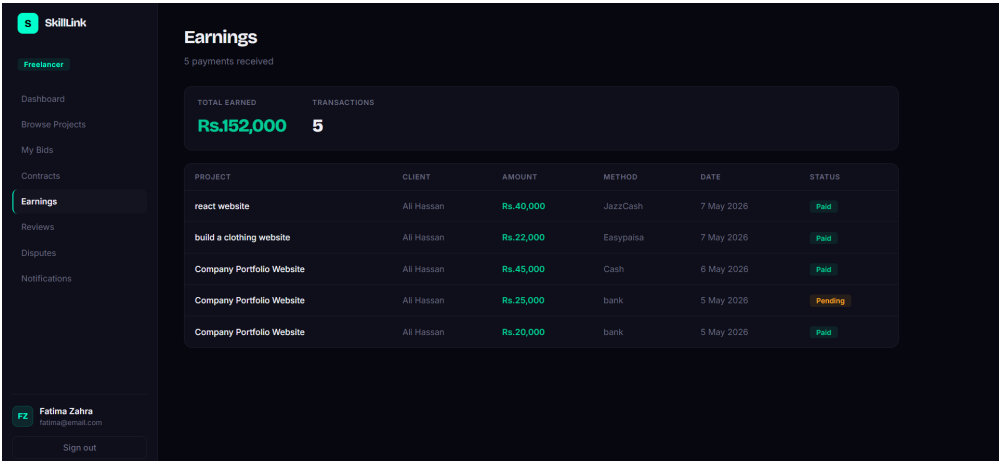


Figure 11.12: Freelancer Earnings Page

11.5 Admin Pages

11.5.1 Admin Dashboard

The Admin Dashboard provides an overview of the entire system with navigation links to all read-only data views.

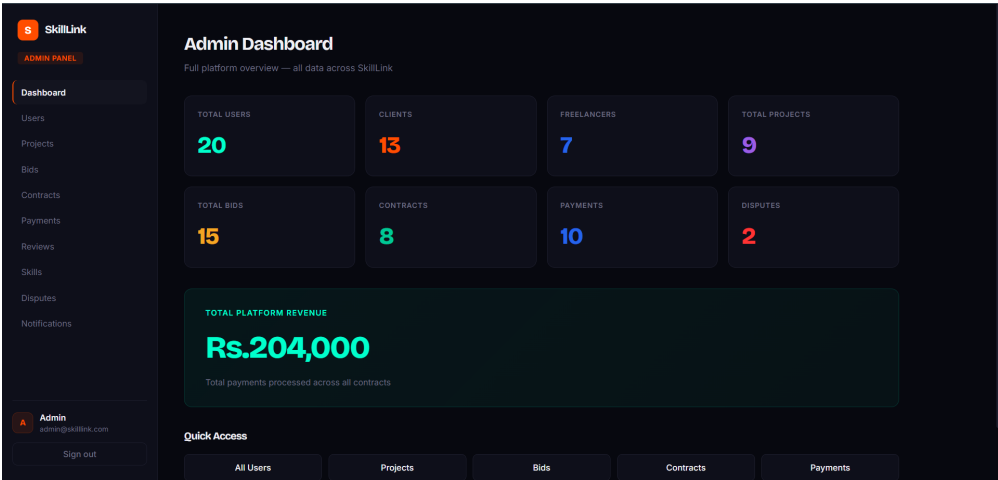


Figure 11.13: Admin Dashboard

11.5.2 All Users

Displays every registered user across the system in a table with their ID, name, email, and role. Data is fetched from the `/api/admin/users` endpoint which joins `users` with `client_profile` to determine role.

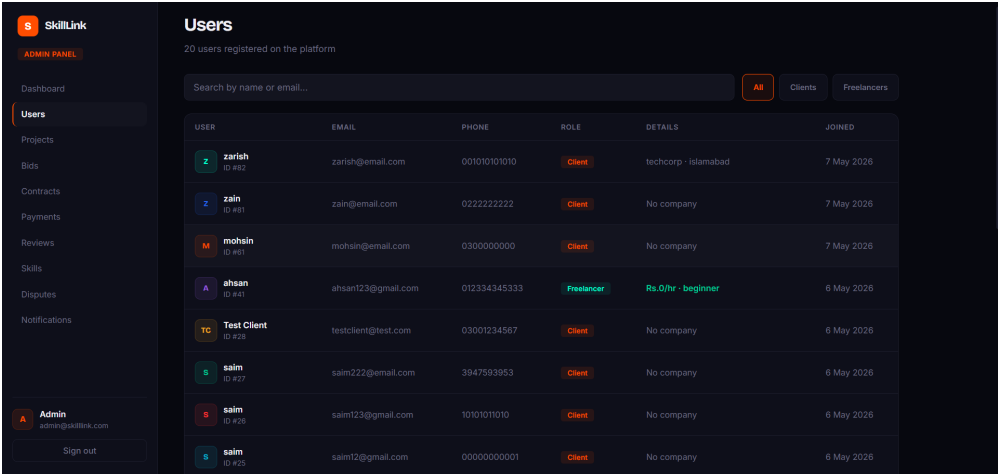


Figure 11.14: Admin — All Users

11.5.3 All Projects

Shows every project in the system with the client name, budget, and status. Uses the same JOIN query as the public projects endpoint but with no status filter.

SkillLink
ADMIN PANEL

Dashboard
Users
Projects
Bids
Contracts
Payments
Reviews
Skills
Disputes
Notifications

Admin
admin@skillink.com
Sign out

Projects
9 projects on the platform

Search by title or client... **All** Open In Progress Completed

PROJECT	CLIENT	CATEGORY	BUDGET	DEADLINE	BIDS	STATUS
abc ID #B1	Ali Hassan TechCorp Lahore	—	Rs. 23,000	11 Nov 2000	1	In Progress
react website ID #A1	Ali Hassan TechCorp Lahore	—	Rs. 30,000	11 Nov 2002	1	Completed
build a clothing website ID #D1	Ali Hassan TechCorp Lahore	—	Rs. 20,000	10 Oct 2026	1	Completed
Social Media Marketing Campaign ID #B	Hamza Sheikh BuildIt Peshawar	Marketing	Rs. 20,000	4 Jun 2026	3	Open
Blog Content for Tech Website ID #B	Ayesha Tariq MediaPro Lahore	Writing	Rs. 12,000	15 May 2026	2	Completed
Mobile App for Food Delivery ID #A	Ahmed Raza Digital Innovations	Mobile Development	Rs. 80,000	19 Jun 2026	2	In Progress
Sales Data Dashboard ID #3	Bilal Ahmed DataGrok	Data Science	Rs. 35,000	26 May 2026	1	In Progress
Logo and Branding Package ID #2	Sara Khan Creative Hub	Design	Rs. 15,000	19 May 2026	2	Completed

Figure 11.15: Admin — All Projects

11.5.4 System Summary

The System Summary page displays key aggregate statistics across all 15 tables: total users, total projects, total contracts, total revenue collected, and the number of open disputes. Data comes from the `/api/admin/summary` endpoint.

TOTAL USERS
20

CLIENTS
13

FREELANCERS
7

TOTAL PROJECTS
9

TOTAL BIDS
15

CONTRACTS
8

PAYMENTS
10

DISPUTES
2

TOTAL PLATFORM REVENUE
Rs. 204,000
Total payments processed across all contracts

Figure 11.16: Admin — System Summary

Summary: The SkillLink interface covers all three user roles across 17 pages. Every page communicates with the Oracle XE backend exclusively through the Node.js REST API. No page accesses the database directly. Role-based routing ensures each user only sees pages relevant to their account type.

Chapter 12

Conclusion and Future Work

12.1 Conclusion

SkillLink was built as a complete, fully working freelance marketplace system covering every layer from the relational database up to the user interface. The project started with a real-world domain, identified every entity and relationship within it, and translated that understanding into a normalized Oracle SQL database, a Node.js REST API, and a React.js frontend.

At the database level, all 15 tables satisfy the Third Normal Form. The design correctly applies disjoint-total specialization for the Users entity, uses weak entities for Bids and Reviews, and resolves three many-to-many relationships through junction tables. Every table is linked through foreign keys and every business rule that can be enforced at the database level is enforced there rather than left to the application.

At the backend level, the 35 API endpoints cover full CRUD operations for every core table. Three multi-table transactions protect the most critical operations in the system: user registration, bid acceptance, and payment recording. Role-based access control separates what clients, freelancers, and administrators can do at both the middleware and route levels.

At the frontend level, 17 React pages give all three roles a working interface. Every page fetches data exclusively from the backend API with a JWT token attached automatically. No page touches the database directly.

The project demonstrates in practice how the concepts covered in CSC 220 connect into a real system. Specialization is not just a diagram concept — it is two subtype tables sharing a primary key with the users table. Normalization is not just a rule — it is the reason a freelancer’s hourly rate is stored in one place and never duplicated.

A transaction is not just a keyword — it is what ensures that when a bid is accepted, the contract is always created and the project status is always updated together or not at all.

Final Summary: 15 Oracle tables, 11 sequences, 35 API endpoints, 17 frontend pages, 3 multi-table transactions, and 3 user roles — all connected into one working system.

12.2 Challenges Faced

Several practical challenges came up during development that are worth documenting.

12.2.1 Oracle Sequence Management

Unlike MySQL which has `AUTO_INCREMENT`, Oracle XE requires a separate sequence object for every table that needs an auto-generated ID. Managing 11 sequences and remembering to call `seq_name.nextval` in every `INSERT` statement added overhead that would not exist in other database engines.

12.2.2 Foreign Key Insertion Order

Oracle enforces foreign key constraints immediately. This meant every `INSERT` and every `CREATE TABLE` had to be written in strict dependency order. A single out-of-order statement would cause the entire script to fail. Building the cleanup and rebuild scripts in the correct reverse order was equally important.

12.2.3 Multi-Table Transactions in Node.js

The `oracledb` package does not auto-commit by default when a connection pool is used. Writing the bid acceptance transaction correctly — getting a raw connection, setting `autoCommit: false`, executing four statements, committing, and releasing the connection inside a `finally` block — required careful handling to avoid connection leaks.

12.2.4 JWT Role Detection at Login

Determining whether a logged-in user is a client, freelancer, or admin required a `LEFT JOIN` across three tables at login time. The `CASE` expression in the login query that checks which subtype table contains the user's ID was the cleanest solution, but it took iteration to get right.

12.2.5 Long SQL Lines in LaTeX

Getting the `listings` package to wrap long SQL lines inside the page margins required configuring `breaklines=true`, `columns=fixed`, `breakindent=0pt`, and `postbreak` together. Using only `breaklines=true` was not sufficient on its own.

12.3 Future Enhancements

The current version of SkillLink is a complete working system, but several features were deliberately left out of scope for this submission and are planned as future enhancements.

Table 12.1: Planned Future Enhancements

Feature	Description
Oracle Triggers	Add BEFORE INSERT triggers to auto-assign sequence values so INSERT statements do not need to call <code>seq_name.nextval</code> explicitly
Stored Procedures	Move complex multi-table logic such as bid acceptance and payment recording into Oracle stored procedures callable from the backend
Real Payment Gateway	Integrate JazzCash or Easypaisa sandbox APIs to process actual payments instead of recording them as data entries
File Upload	Replace the metadata-only <code>project_files</code> table with actual file storage using a cloud bucket such as AWS S3 or Cloudinary
Real-Time Notifications	Replace the polling-based notification system with WebSocket connections so freelancers receive bid updates instantly
Search and Filtering	Add full-text search across project titles and descriptions using Oracle Text, and allow filtering by category, budget range, and deadline
Password Hashing	Replace plaintext password storage with bcrypt hashing in the registration endpoint
Admin Moderation	Give the admin role the ability to suspend users, close disputes, and override contract statuses rather than read-only access
Mobile App	Build a React Native mobile version of the frontend sharing the same backend API
Analytics Dashboard	Add an admin analytics page with charts showing platform growth, revenue trends, and most active freelancers over time

12.4 Learning Outcomes

This project gave practical experience with every major topic in the CSC 220 curriculum:

- Designing a complete ER diagram for a real-world domain with specialization, weak entities, and M:N relationships
- Converting an ER diagram into a full relational schema with correct primary keys, foreign keys, and constraints
- Writing and executing DDL scripts in Oracle SQL including sequences, CREATE TABLE, and constraint definitions
- Inserting realistic sample data in the correct dependency order across 15 tables
- Writing complex SQL queries using INNER JOIN, LEFT JOIN, aggregate functions, GROUP BY, HAVING, CASE, and subqueries
- Proving that all 15 tables satisfy 1NF, 2NF, and 3NF with concrete examples
- Building a full REST API in Node.js with Oracle XE as the database and JWT authentication
- Implementing multi-table transactions with proper rollback handling
- Building a role-based React frontend with protected routing

Submitted by: Zairish Fatima (01-135241-052), Maryam Qadir (01-135241-028), Saim Zafar (01-135241-045)

Course: CSC 220 — Database Management System

Instructor: Ms. Mehwish Pervaiz

Institution: Bahria University Islamabad